Multiplayer Blackjack

Software Requirements Specification

Revision History

Date	Revision	Description	Author
02/15/2026	1.0	Initial Version	Michael, Curtis, Emma
02/25/2026	2.0	Started requirement list (2.3), added modules, filled out first three sections of SRS	Curtis, Michael, Emma
02/26/2026	2.5	Filled functional requirements (3.1)	Emma
02/27/2026	2.6	Added to functional requirements (3.1)	Emma
02/28/2026	2.7	Modified dependencies (2.5), modified and added to functional requirements (3.1), created non-functional requirements (4), resolved a contradiction from high-level features and player module requirements (2.3)	Curtis
03/01/2026	2.8	Added pages for references, modified 2.3, 3.1.7, 3.1.2, and 3.1.3	Curtis, Emma
03/03/2026	2.9	Replaced mentions of “boot” with “shoe”	Curtis
03/04/2026	2.10	Updated 1.2, 2.3, 3.1.4, 3.1.5, 4.2, and 4.3	Michael
03/05/2026	2.11	Removed mention of exact time for turns	Curtis
03/05/2026	3.0	Added Use Case Specifications, UML Use Case Diagram, UML Class Diagram, UML Sequence Diagrams	Emma, Curtis, Michael
04/09/26	4.0	Diagrams updated from design phase	Curtis

Table of Contents

1. Purpose **4**
1.1. Scope 4
1.2. Definitions, Acronyms, Abbreviations 4
1.3. References 4
1.4. Overview 4
2. Overall Description **5**
2.1. Product Perspective 5
2.2. Product Architecture 5
2.3. Product Functionality/Features 5
2.4. Constraints 6
2.5. Assumptions and Dependencies 6
3. Specific Requirements **7**
3.1. Functional Requirements 7
3.1.1. Common Requirements: 7
3.1.2. Server Module Requirements: 7
3.1.3. Client Module Requirements: 7
3.1.4. Dealer Module Requirements: 7
3.1.5. Player Module Requirements 8
3.1.6. Game Module Requirements: 8
3.1.7. Table Module Requirements: 9
3.2. External Interface Requirements 10
3.3. Internal Interface Requirements 10
4. Non-Functional Requirements **11**
4.1. Security and Privacy Requirements 11
4.2. Environmental Requirements 11
4.3. Performance Requirements 11
Use Case Specifications **12**
UML Use Case Diagram **22**
UML Class Diagram **23**
UML Sequence Diagrams **24**

1. Purpose

This document outlines the requirements for the Multiplayer Blackjack program.

1.1. Scope

This document will catalog the user, system, and hardware requirements for the Multiplayer Blackjack program. It will not, however, document how these requirements will be implemented.

1.2. Definitions, Acronyms, Abbreviations

Term	Definition
Balance	How much currency a player user can use to bet with
Bust	When the cards total surpasses 21
Shoe	A set of cards composed of multiple decks shuffled together
Hand	The cards a player possesses in a round
Hit	To request an additional card to add to the current hand
Hole card	The first card drawn by the dealer, which is dealt face down
Rank	The numerical point value of a card
Stand	To keep the current hand during a turn

1.3. References

Use Case Specifications – Pages 12-21

UML Use Case Diagram – Page 22

UML Class Diagrams – Page 23

UML Sequence Diagrams – Pages 24-26

1.4. Overview

The Multiplayer Blackjack program simulates the experience of walking into a casino and playing blackjack. Players join a table with a dealer and bet from their balance. Players can hit or stand to get their score as close to 21 as possible without going over. If a player's score is better than the dealer's, they win their bets, but if the dealer has a better score, the player loses their bets.

2. Overall Description

2.1. Product Perspective

The Multiplayer Blackjack program is perfect for casinos that wish to provide their users with a mobile experience for this classic card game.

2.2. Product Architecture

The system will be organized into 6 major modules: The Server Module, The Client Module, The Dealer Module, The Player Module, The Game Module, and The Table Module.

2.3. Product Functionality/Features

The high-level features of the system are as follows (see section 3 of this document for more detailed requirements that address these features):

2.3.1. Users are split into two roles: dealers and players
2.3.2. Client connects to server
2.3.3. User logs into the client application with username and password
2.3.4. The system will have a main menu
2.3.5. From the main menu, the user can quit, join a table, or check the balance
2.3.6. Players can deposit or withdraw from their balance
2.3.7. Each table can take 6 players and 1 dealer
2.3.8. The Blackjack game starts when a table has at least 1 player and 1 host
2.3.9. Players choose what they bet each game under a time limit
2.3.10. Bets are validated
2.3.11. All users draw two cards, starting with the dealer, going clockwise, one at a time
2.3.12. The turn order goes counterclockwise, starting with the player closest to the dealer
2.3.13. Users can see every users' cards, excluding the dealer's hole card
2.3.14. Users can hit or stand during their turn
2.3.15. Players who stand cannot take another turn
2.3.16. Users have a time limit on their turns
2.3.17. Users draw from the shoe, which consists of six decks of standard playing cards
2.3.18. Users whose ranks add to over 21 bust, with players losing bets and the dealer causing the whole table to win theirs
2.3.19. Dealers must stand at least 17
2.3.20. After all players stand or bust, the dealer hits until they must stand or bust
2.3.21. Users whose ranks are below the dealer's loses their bets
2.3.22. Users whose ranks are above the dealer's win their bets
2.3.23. Users whose ranks match the dealer's keep their bets
2.3.24. Blackjacks are an instant win for users, prioritizing dealers
2.3.25. Aces count as either 11 or, if 11 would cause a bust, 1 in terms of rank
2.3.26. Face cards all count as 10 in terms of rank
2.3.27. Players can leave tables at any time
2.3.28. Dealers can leave after a blackjack game finishes
2.3.29. Users who disconnect may be timed out from joining another table to prevent players not losing their bets
2.3.30. Users who join mid-game can only start playing when the next game starts
2.3.31. Games start every few seconds at a table if none are in progress
2.3.32. Servers should keep a record of players' balances, usernames, passwords, and roles
2.3.33. Players/Dealer have a time limit to make choices

2.4. Constraints

2.4.1. The program must be programmed in Java
2.4.2. The program must use TCP/IP for communication
2.4.3. The program must have a GUI.
2.4.4. The program must consist of a server application and client application.
2.4.5. The program should have no web component.
2.4.6. No databases should be used for saving information

2.5. Assumptions and Dependencies

2.5.1. The program is assumed to run on hardware that supports a mouse and keyboard for user input.

2.5.2. The program is assumed to run on hardware that tracks time.

2.5.3. The server is assumed to run on hardware that can support the connection of multiple clients.

2.5.4. A client is assumed to run on hardware that supports network connection.

3. Specific Requirements

3.1. Functional Requirements

3.1.1. Common Requirements:

3.1.1.1. There are two types of users of the system: dealer and player.
3.1.1.2. The system stores account data (username, password, balance).
3.1.1.3. Passwords are encrypted in storage.

3.1.2. Server Module Requirements:

3.1.2.1. The server allows multiple client connections simultaneously.

3.1.2.2. The server stores account data when users register.

3.1.2.3. The server validates users when they sign in.

3.1.2.4. The server stores users' balances.

3.1.2.5. The server picks a random table that has the least number of players for a player to join.

3.1.2.6. The server hosts tables and their blackjack games.

3.1.2.7. The server validates bets by checking users' balances.

3.1.2.8. The tracks bet amounts for the round.

3.1.2.9. The server keeps track of the shoe and the cards players hold and draw.

3.1.2.10. The server issues time outs to players who disconnect from the server or table mid-game to prevent players from keeping lost bets.

3.1.2.11. The server informs the client of any validation errors or time outs.

3.1.3. Client Module Requirements:

3.1.3.1. The client connects to the server on start up.

3.1.3.2. Users navigate the client application using a GUI.

3.1.3.3. The client can receive player balance from the server.

3.1.3.4. The client can send back a modified player balance to the server to be stored.

3.1.3.5. The client can request the server to find a table for the client to join.

3.1.3.6. During a blackjack round, the client receives information from the server on what cards a user is holding and what the dealer's non-hole card is.

3.1.3.7. The client allows the user to place a bet by selecting chips.

3.1.3.8. The client tells the server what a user has bet.

3.1.3.9. The client tells the server when a user has stood or hit.

3.1.3.10. When the client is informed of a validation error or time out, that information is displayed to the user.

3.1.4. Dealer Module Requirements:

3.1.4.1. New dealer accounts can be created.
3.1.4.2. Dealers sign into their account using username and password.

3.1.4.3. Failed dealer login alerts user and prompts again.

3.1.4.4. A dealer account can only be signed in on one device at a time.

3.1.4.5. At the beginning of a round, the dealer first deals two cards to each player (all face up), then two to themself (one face up and one face down).

3.1.4.6. After all players have played, the dealer plays following fixed rules.

3.1.4.7. The dealer must hit on a hand of 16 or less and stand on a hand of 17 or more.

3.1.4.8. The dealer must continue to hit until their card total adds to 17 or more.

3.1.4.9. Once the dealer finishes their turn, the round is completed.

3.1.4.10. The dealer reveals their hand at the end of the round.

3.1.5. Player Module Requirements

3.1.5.1. New player accounts can be created.

3.1.5.2. Players sign into their account using their username and password.

3.1.5.3. Failed player login alerts the user and prompts again.

3.1.5.4. A player account can only be signed in on one device at a time.

3.1.5.5. Players have a balance attached to their account.

3.1.5.6. Players can deposit into and withdraw from their balance. This is entered manually by the player.

3.1.5.7. The player must place a bet at the beginning of each round.

3.1.5.8. A player places a bet by selecting chips.

3.1.5.9. When a player places a bet, the bet amount is removed from the player's balance.

3.1.5.10. A player cannot bet more than what is available in their balance. Bets are validated.

3.1.5.11. During a round of blackjack, a player has a hand made up of cards.

3.1.5.12. A player starts every round with two cards that are visible to everyone.

3.1.5.13. A player’s turn times out after some time. A turn timeout is an automatic stand.

3.1.5.14. During a player's turn, they can either hit (get another card to add to their hand) or stand (stay with their current hand).

3.1.5.15. During a player’s turn, they can continue to hit until their card total surpasses 21, at which point they have busted.

3.1.5.16. Cards received with hits are visible to everyone.

3.1.5.17. A player's turn ends once they either stand or bust.

3.1.5.18. A bust is an automatic loss.

3.1.5.19. At the end of a round a player loses or gains from their bet depending on how their hand compares to the dealer's hand.

3.1.5.20. If the player wins, the winnings are added to the player’s balance.

3.1.6. Game Module Requirements:

3.1.6.1. A card deck contains 52 cards.

3.1.6.2. A card deck is made up of 4 suits and 13 ranks.

3.1.6.3. A card deck can be shuffled.

3.1.6.4. A shoe consists of 6 mixed card decks.

3.1.6.5. The goal of the game blackjack is to get the card total of your hand as close to 21 as possible without going over.

3.1.6.6. Each player plays individually against the dealer.

3.1.6.7. The turn order goes counterclockwise, starting with the player closest to the dealer

3.1.6.8. Number cards 2-10 count as their respective numbers.

3.1.6.9. Face cards (king, queen, jack) each count as 10.

3.1.6.10. Ace cards count as either a 1 or an 11 depending on what is best for the hand.

3.1.6.11. A Blackjack is when the first two cards in a hand are an ace and a 10-value card.

3.1.6.12. A hand busts when its total surpasses 21.

3.1.6.13. The blackjack round cycle is betting, initial dealing, player turns, dealer turn, then bet resolution.

3.1.6.14. Bets are placed with chips.

3.1.6.15. Chips are tokens that equate to either $1, $5, $25, $50, $100, or $500.

3.1.6.16. Players play first.

3.1.6.17. The dealer plays last.

3.1.6.18. At the end of a round, hands are compared and bets are settled.

3.1.6.19. When the player's hand is larger than the dealer's hand, the player wins 2x their bet (1:1 winning).

3.1.6.20. When the dealer busts, the player wins 2x their bet (1:1 winning).

3.1.6.21. When the player busts, the player loses their bet.

3.1.6.22. When the dealer’s hand is larger than the player’s hand, the player loses their bet.

3.1.6.23. When the player’s hand and the dealer’s hand are the same, the player gets their bet back.

3.1.6.24. When the player has a Blackjack and does not tie with the dealer, the player wins 2.5x their bet (3:2 winning).

3.1.7. Table Module Requirements:

3.1.7.1. A table consists of at most 1 dealer.

3.1.7.2. A table can host 0-6 players.

3.1.7.3. A round of blackjack at an empty table begins after a few seconds after a player and a dealer join the table.

3.1.7.4. A table hosts consecutive rounds while the table is not empty.

3.1.7.5. A player cannot join a full table (6 players).

3.1.7.6. A new table automatically opens when all current tables are full of either players or dealers.

3.1.7.7. A dealer cannot join a table with a dealer already present in it.

3.1.7.8. Joining a table, a player must wait until the next round to play.

3.1.7.9. Multiple tables can be active at once.

3.1.7.10. Blackjack rounds start every few seconds at a non-empty table if none are in progress.

3.1.7.11. Users can leave a table.

3.1.7.12. Players can leave a table at any time.

3.1.7.13. Dealers can only leave while no round is in play.

3.1.7.14. Dealers can queue their leave mid-game.

3.1.7.15. Dealers who queued their leave will leave immediately after a game finishes.

3.2. External Interface Requirements

3.2.1. There will be a graphical user interface.
3.2.2. There will be a sign in menu. From this menu a user can sign into an existing account, register as a player, or register as a dealer.
3.2.3. If the user chooses to sign in they are taken to a menu that asks for a username and password for an existing account.
3.2.4. If the user chooses to register they are taken to a menu that asks for a username and password to register a new account.
3.2.5. After players sign in, there will be a main player menu. From this menu players can view their account balance, add funds to their account, cash out their account, join a blackjack table, or log out.
3.2.6. Players will be able to see their current updated balance while playing a game.
3.2.7. At the beginning of a round, a player places a bet by selecting from chips displayed.
3.2.8. During a player's turn, the player will have a button to hit and a button to stand.
3.2.9. At the end of a round, a player's outcome is displayed (win, loss, tie).
3.2.10. After dealers sign in, there will be a main dealer menu. From this menu dealers can host a table or log out.
3.2.11. The system must run on hardware with networking abilities, keyboard, mouse, and display.
3.2.12. Depositing or withdrawing from the players' balance is handled by a banking service.

3.3. Internal Interface Requirements

3.3.1. Server and clients will communicate over TCP/IP.
3.3.2. User actions passed from client to server.
3.3.3. Clients cannot modify game data.
3.3.4. Balance modifications are concurrent.
3.3.5. No component shall directly modify another component's internal data.

4. Non-Functional Requirements

4.1. Security and Privacy Requirements

4.1.1. Passwords are encrypted in storage
4.1.2. User balance modifications and betting is validated

4.2. Environmental Requirements

4.2.1. No databases should be used for saving information
4.2.2. The client requires an active internet connection to communicate with the server
4.2.3. The clients and servers are required to have a Java Runtime Environment

4.3. Performance Requirements

4.3.1. Users should be able to use their allotted decision-making time within a minute
4.3.2. The server must process a user's input action, such as a hit, within a second
4.3.3. The server should be able to support up a scalable amount of tables at a time

Use Case Specifications

Use Case ID: USER-01
Use Case Name: Register Player
Relevant Requirements:
- 3.1.4.1
- 3.1.5.1

Primary Actor: Player
Pre-conditions:
- There is no account currently logged in on the user's system.

Post-conditions:
- The user has created a player account.
- The user is logged into the new player account.

Basic Flow or Main Scenario:
1. The GUI prompts for an action on OpenScene(sign in, register as player, or register as dealer).
2. The user chooses to register as a player.
3. GUI is updated to PlayerRegisterScene.
4. The GUI prompts for a username and password to create an account with.
5. The user enters a username and password.
6. The GUI validates the username and password.
7. Message of type Register sent from Client to Server holding username, password and account credentials.
8. Client handler takes username, password, and credentials and makes a new account, returning a message of type Registered indicating successful account registry and log in to Client.
9. Client GUI is updated to PlayerMainMenuScene.

Extensions or Alternate Flows:
Extension 1: Username is Invalid
- 5.1. The system informs the user that the username is invalid (not available).
- 5.2. Process returns to step 3.

Extension 2: Password is Invalid
- 5.1. The system informs the user that the password is invalid (doesn't meet requirements).
- 5.2. Process returns to step 3.

Exceptions:
Exception 1: Invalid Input
Related Use Cases: N/A

Use Case ID: USER-02
Use Case Name: Register Dealer
Relevant Requirements:
- 3.1.4.1
- 3.1.5.1

Primary Actor: Dealer
Pre-conditions:
- There is no account currently logged in on the user's system.

Post-conditions:
- The user has created a dealer account.
- The user is logged into the new dealer account.

Basic Flow or Main Scenario:
1. The GUI prompts for an action on OpenScene(sign in, register as player, or register as dealer).
2. The user chooses to register as a dealer.
3. GUI is updated to DealerRegisterScene.
4. The GUI prompts for a username and password to create an account with.
5. The user enters a username and password.
6. The GUI validates the username and password.
7. Message of type Register sent from Client to Server holding username, password and account credentials.
8. Client handler takes username, password, and credentials and makes a new account, returning a message of type Registered indicating successful account registry and log in to Client.
9. Client GUI is updated to DealerMainMenuScene.

Extensions or Alternate Flows:
Extension 1: Username is Invalid
5.3. The system informs the user that the username is invalid (not available).
5.4. Process returns to step 3.
Extension 2: Password is Invalid
5.3. The system informs the user that the password is invalid (doesn't meet requirements).
5.4. Process returns to step 3.

Exceptions:
Exception 1: Invalid Input
Related Use Cases: N/A

Use Case ID: USER-03
Use Case Name: Log in
Relevant Requirements:
- 3.1.4.2
- 3.1.5.2

Primary Actor: User (Player or Dealer)
Pre-conditions:
- There is no account currently logged in on the user's system.
- The account exists.

Post-conditions:
- The user is logged in on their system.

Basic Flow or Main Scenario:
1. The GUI prompts for an action on OpenScene (sign in, register as player, or register as dealer).
2. The user chooses to sign in.
3. GUI is updated to LogInScene.
4. The GUI prompts for a username and password.
5. The user enters a username and password.
6. The GUI validates the username and password.
7. Message of type LogIn sent from Client to Server holding the entered username and password, and account credentials.
8. ClientHandler takes username, password, and credentials and attempts logIn.
9. Account returns true boolean indicating successful login.
10. ClientHandler returns a message of type SuccessfulLogIn to Client.
11. Client GUI is updated to PlayerMainMenuScene.

Extensions or Alternate Flows:
Alternative 1: Login Failure
9. Account returns false boolean indicating failed login.
10. ClientHandler returns a message of type Error with ErrorType of either InvalidUsernameOrPassword or InvalidCredentials to Client.
11. The GUI notifies the user of the failed login.
12. The GUI prompts for a username and password.
13. Repeat from step 6.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-01: Register Player
- USER-02: Register Dealer

Use Case ID: USER-04
Use Case Name: Log Out
Relevant Requirements:
- 3.1.4.2
- 3.1.5.2

Primary Actor: User (Player or Dealer)
Pre-conditions:
- The account exists. and is currently logged in.

Post-conditions:
- The user is logged out.

Basic Flow or Main Scenario:
1. The system prompts for an action (sign in, register as player, or register as dealer).
2. The user chooses to log out.
3. Client sends message of type LogOut to Server, indicating signed in user wants to log out.
4. ClientHandler calls logOut in account which updates signedIn boolean attribute to false.
5. Control returns to Client.
6. Client GUI is updated to OpenScene.

Extensions or Alternate Flows:
N/A
Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-03: Log In

Use Case ID: PLAYER-01
Use Case Name: Check Balance
Relevant Requirements:
- 3.1.5.5
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is displayed to the player.

Basic Flow or Main Scenario:
1. PlayerMainMenuScene prompts action from the player(view account balance, join blackjack table, or log out).
2. The player chooses to view balance.
3. The Client requests the current balance attached to the signed in player's account from Server via message of type BalanceRequest.
4. ClientHandler fetches account balance.
5. Balance returned to Client via a message of type BalanceView.
6. GUI updates to BalanceScene, displaying the player's current balance.

Extensions or Alternate Flows: N/A
Exceptions: N/A
Related Use Cases:
- USER-03: Log in

Use Case ID: PLAYER-02
Use Case Name: Deposit
Relevant Requirements:
- 3.1.5.6
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is updated including the new deposit.

Basic Flow or Main Scenario:
1. PlayerMainMenuScene prompts action from the player(view account balance, join blackjack table, or log out).
2. The player chooses to view account balance.
3. The Client requests the current balance attached to the signed in player's account from Server via message of type BalanceRequest.
4. ClientHandler fetches account balance.
5. Balance returned to Client via a message of type BalanceView.
6. GUI updates to BalanceScene, displaying the player's current balance.
7. GUI prompts action from the player(deposit, withdraw, back).
8. The Player chooses to deposit.
9. GUI prompts deposit amount from player.
10. The player enters a deposit amount.
11. GUI validates the entered deposit amount.
12. The Client requests a balance deposit from the Server via a message of type DepositRequest.
13. ClientHandler adds the given amount from the player's account balance.
14. New balance returned to Client via a message of type Deposit.
15. GUI updates BalanceView to display new balance.

Extensions or Alternate Flows:
Extension 1: Invalid deposit input
- 7.1. The system informs the user that the input is invalid.
- 7.2. Process returns to step 5.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-03: Log in

Use Case ID: PLAYER-03
Use Case Name: Withdraw
Relevant Requirements:
- 3.1.5.6
- 3.2.5

Primary Actor: Player
Pre-conditions:
- The player is logged in.

Post-conditions:
- The player's current balance is updated minus withdrawal.

Basic Flow or Main Scenario:
1. PlayerMainMenuScene prompts action from the player(view account balance, join blackjack table, or log out).
2. The player chooses to view account balance.
3. The Client requests the current balance attached to the signed in player's account from Server via message of type BalanceRequest.
4. ClientHandler fetches account balance.
5. Balance returned to Client via a message of type BalanceView.
6. GUI updates to BalanceScene, displaying the player's current balance.
7. GUI prompts action from the player(deposit, withdraw, back).
8. The Player chooses to withdraw.
9. GUI prompts deposit amount from player.
10. The player enters a withdrawal amount.
11. GUI validates the entered deposit amount (amount<=current balance).
12. The Client requests a balance withdrawal from the Server via a message of type WithdrawRequest.
13. ClientHandler removes the given amount from the player's account balance.
14. New balance returned to Client via a message of type Withdraw.
15. GUI updates BalanceView to display new balance.

Extensions or Alternate Flows:
Extension 1: Invalid deposit input
7.3. The system informs the user that the input is invalid.
7.4. Process returns to step 5.

Exceptions:
Exception 1: Input Invalid
Related Use Cases:
- USER-03: Log in

Use Case ID: PLAYER-04
Use Case Name: “Join table”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.7
Primary Actor: Player or dealer
Pre-conditions: The user, whether it be a player or dealer, is logged into the program. The user also does not have a time out.
Post-conditions: The user is at a table. A table may be opened.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds an available table;
5. The player joins that table.

Extensions or Alternate Flows:
Extension 1: no table available

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds no available table, so it creates a new table;
5. The player joins the new table.

Alternate Flow 1: Dealer case

1.The dealer is logged into the Multiplayer Blackjack program;
2. The dealer requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds an available table;
5. The dealer joins that table.

Extension 2: Dealer case, no table available

1.The dealer is logged into the Multiplayer Blackjack program;
2. The dealer requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server finds no available table, so it creates a new table;
5. The player joins the new table.

Extension 3: Player timed out

1.The player is logged into the Multiplayer Blackjack program;
2. The player requests to play blackjack from the client UI;
3. The client requests the server find an available table;
4. The server informs the client that the player is timed out for some time;
5. The client UI displays that the player is timed out and how long until they are free to join again.

Exceptions: The player is timed out
Related Use Cases: USER-03: Log in, PLAYER-05: Bet

Use Case ID: PLAYER-05
Use Case Name: “Bet”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.5, 3.1.6
Primary Actor: Player
Pre-conditions: The player is at a table and is currently active in a blackjack game.
Post-conditions: The player has placed a bet, which is subtracted from their balance.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2.The player joins a table;
3. A game of blackjack begins;
4. The player is prompted by the server to place a bet;
5. The player chooses the chips they wish to bet with through the UI;
6. The server checks the player’s balance;
7. The player can afford the bet;
8. The bet is placed, with currency subtracted from the balance equal to the worth of the chips.

Extensions or Alternate Flows:
Extension 1: Invalid bet

1.The player is logged into the Multiplayer Blackjack program;
2.The player joins a table;
3. A game of blackjack begins;
4. The player is prompted by the server to place a bet;
5. The player chooses the chips they wish to bet with through the UI;
6. The server checks the player’s balance;
7. The player cannot afford the bet;
8. The server tells the client that the player cannot place the bet;
9. The process repeats from step 4;
10. If no bet is placed in time, a bet of the lowest amount is made automatically.

Exceptions: The player has insufficient balance or runs out of time.
Related Use Cases:

- USER-03: Log in
- USER-04: Join table
- USER-06: Hit
- USER-07: Stand

Use Case ID: USER-05
Use Case Name: “Leave table”
Relevant Requirements: 3.1.7
Primary Actor: Player or dealer
Pre-conditions: The user is at a table.
Post-conditions: The user is returned to the starting menu or will be returned after a game concludes.
Basic Flow or Main Scenario:

1.The player is logged into the Multiplayer Blackjack program;
2.The player is at a table;
3. The player requests from the client UI to leave the table;
4. The server removes the user from the table; 5. The player is back at the starting menu.

Extensions or Alternate Flows:
Alternate flow 1: Dealer case, game active

1. The dealer is logged into the Multiplayer Blackjack program;
2.The dealer is at a table;
3. The dealer requests from the client UI to leave the table;
4. There is a blackjack game active;
5. The server keeps a record that the dealer wants to leave;
6. After the game ends, the dealer automatically leaves the table;
7. The dealer is back at the starting menu.

Extension 1: Dealer case, no game active

1. The dealer is logged into the Multiplayer Blackjack program;
2. The dealer is at a table;
3. The dealer requests from the client UI to leave the table;
4. There is no blackjack game active;
5. The server removes the dealer from the table;
6. The dealer is back at the starting menu.

Exceptions: N/A
Related Use Cases:

- USER-03: Log in
- USER-04: Join table

Use Case ID: USER-06
Use Case Name: “Hit”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: User is at a table, they are not busted, and it is currently their turn
Post-conditions: The user receives an additional card; if the total exceeds 21, the player busts And the turn ends
Basic Flow or Main Scenario:

1. The user requests an additional card from the Client UI.
2. The Client sends the “Hit” request to the server.
3. The server tells the Table class to execute the pullCard method from the CardDeck.
4. The system adds the card to the player’s hand and updates the hand total.
5. The system displays the new card and updates the total to the player.

Related Use Cases:

- USER-04: Join table
- USER-07: Stand

Use Case ID: USER-07
Use Case Name: “Stand”
Relevant Requirements: 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: The player is at a table, and it’s currently their turn
Post-conditions: The user is disconnected from the server and the application closes
Basic Flow or Main Scenario:

1. The user selects the “Stand” option from the Client UI.
2. The Client sends a standRequest.
3. The Server signals the Table class to trigger the nextTurn method.
4. The system locks the player’s UI for the remainder of the round.

Related Use Cases:

- USER-04: Join table
- USER-06: Hit

Use Case ID: USER-08
Use Case Name: “Exit Program”
Relevant Requirements: 3.1.3, 3.1.4, 3.1.5, 3.1.7
Primary Actor: Player/Dealer
Pre-conditions: The program is currently running.
Post-conditions: The user is disconnected from the server, and the application closes
Basic Flow or Main Scenario:

1. The user selects the “Exit” or “Close” option from the Client UI.
2. The Client sends a leaveRequest to the Server.
3. The Server removes the user from any active tables via the removeUserFromTable method.
4. The Server terminates the connection session.
5. The Client application closes.

Related Use Cases:

- USER-03: Log in
- USER-05: Leave table

Use Case ID: DEALER-01
Use Case Name: “Opens Table”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.7
Primary Actor: Dealer
Pre-conditions: Dealer is logged in,
Post-conditions: A new BlackJack table is created on the server, the table is available for players to join, and the dealer joins the table
Basic Flow or Main Scenario:

1. The user selects the option to open a new table from the Client
2. The Client sends a message and sends it to the Server
3. The Server reads the messages and calls makeTable() to the ClientHandler
4. A new Table instance is created and added to the availableTables on the Server
5. The Server makes the Dealer join at the created table using the addUserToTable(ClientHandler user) method

Related Use Cases:

- USER-03: Log in
- DEALER-03 Host Game
- DEALER-02 Cancels Game

Use Case ID: DEALER-02
Use Case Name: “Cancels Game”
Relevant Requirements: 3.1.7
Primary Actor: Dealer
Pre-conditions: The user is at a table.
Post-conditions: The Dealer and Players return to the starting menu or will be returned and the table’s game will be cancelled and removed from the server
Basic Flow or Main Scenario:

1. The dealer selects the option to cancel the game from the Client UI
2. The Client sends a cancellation request Message to the Server
3. The Server receives a request and calls the removeFromTable() method from the ClientHandler
4. The Server triggers the removeUserFromTable(ClientHandler user) method for the dealer and all seated players
5. The server updates the Client UI to reflect the SceneType change back to the main menu for all disconnected users

Related Use Cases:

- USER-03: Log in
- DEALER-01: Opens Table
- USER-05: Leave Table

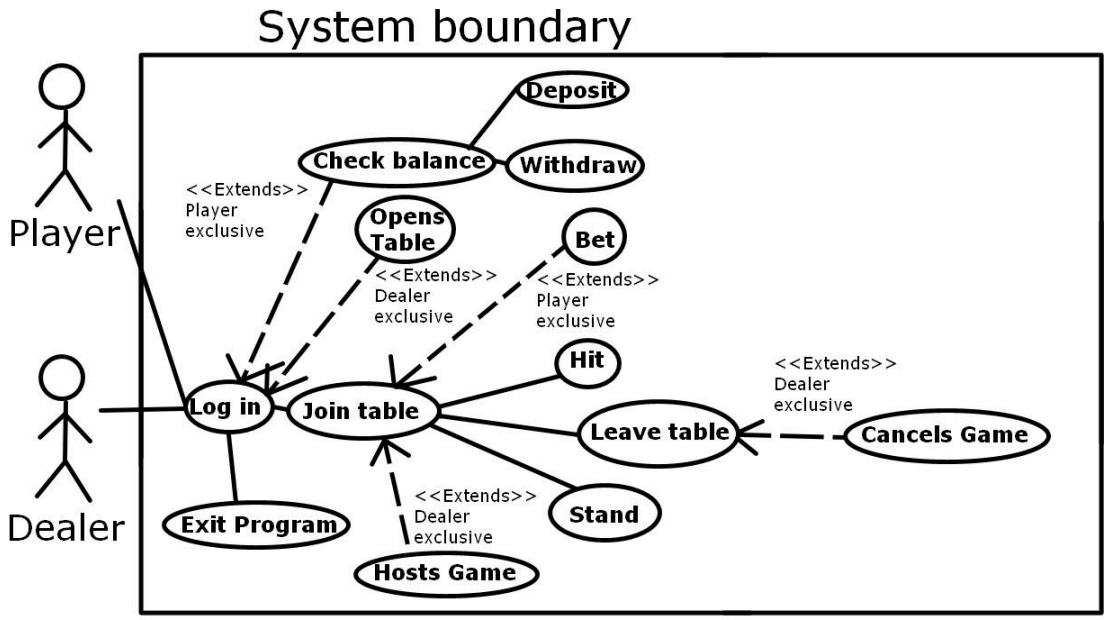
Use Case ID: DEALER-03
Use Case Name: “Hosts Game”
Relevant Requirements: 3.1.2, 3.1.3, 3.1.4, 3.1.5
Primary Actor: Dealer
Pre-conditions: Dealer is logged in,
Post-conditions: A new game of BlackJack becomes active and the server begins polling players for their bets
Basic Flow or Main Scenario:

1. The dealer selects the option to host/start the game from the Client UI
2. The Client sends a request to The Server to initiate the round
3. The Server calls the startGame() method within the Table class
4. The Table’s gameActive boolean attribute is set to true to lock the table state
5. The Server triggers the askForBets() method via the ClientHandler to prompt all players to input their wager

Related Use Cases:

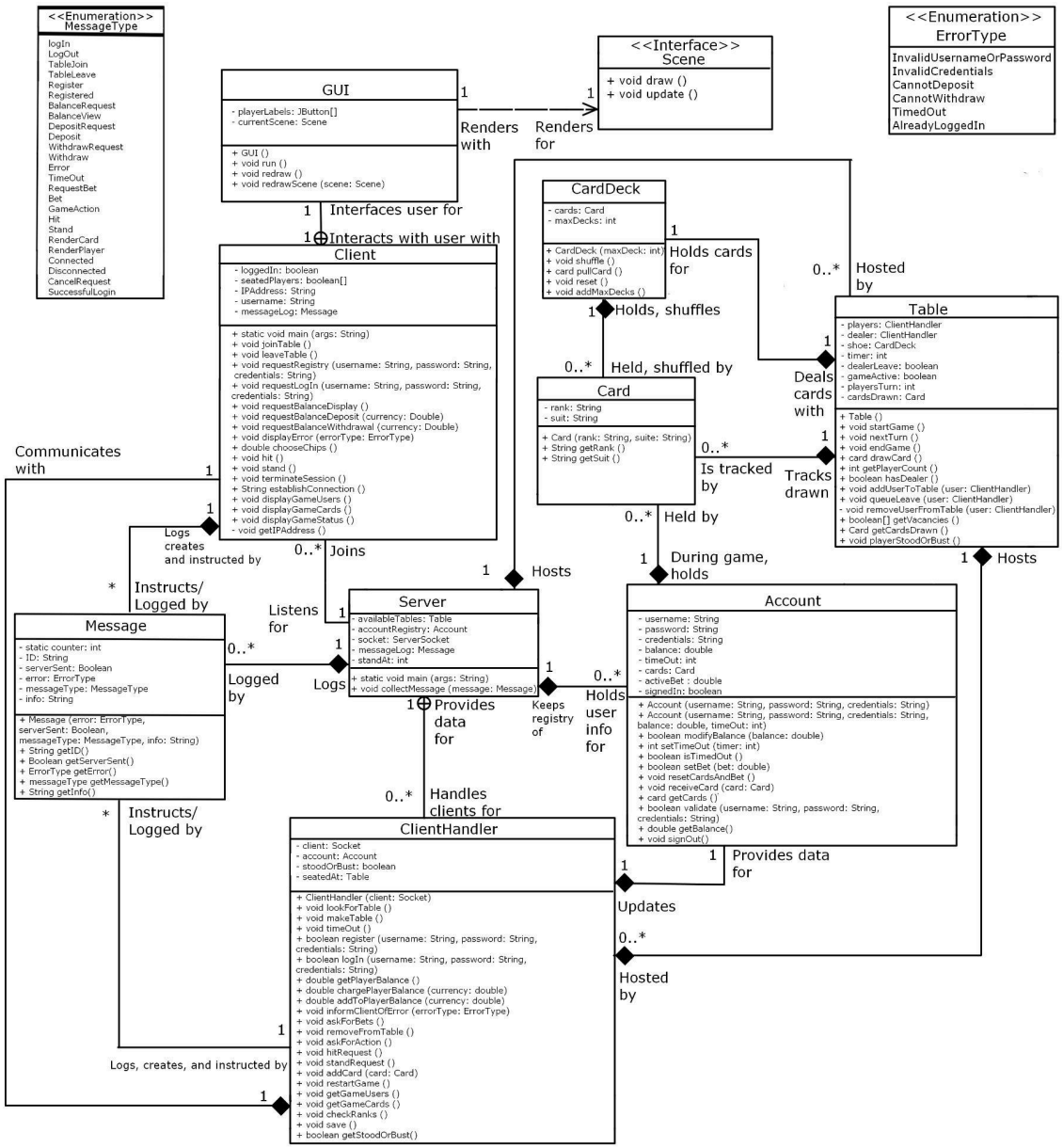
- USER-03: Log in
- DEALER-01: Opens Table
- PLAYER-05: Bet

UML Use Case Diagram

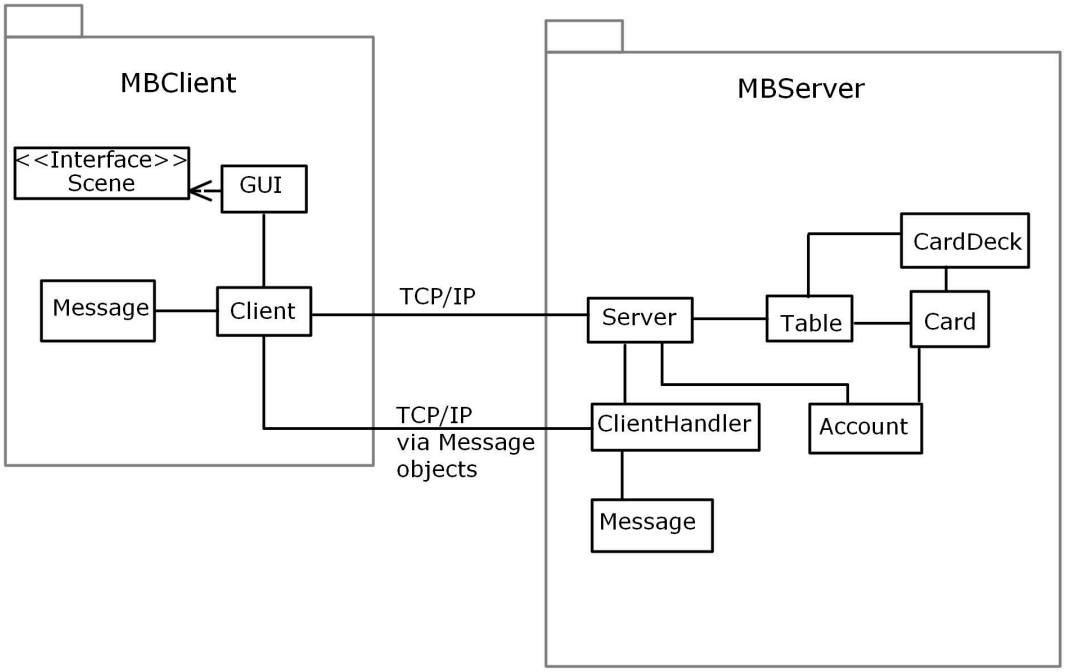


UML Class Diagrams

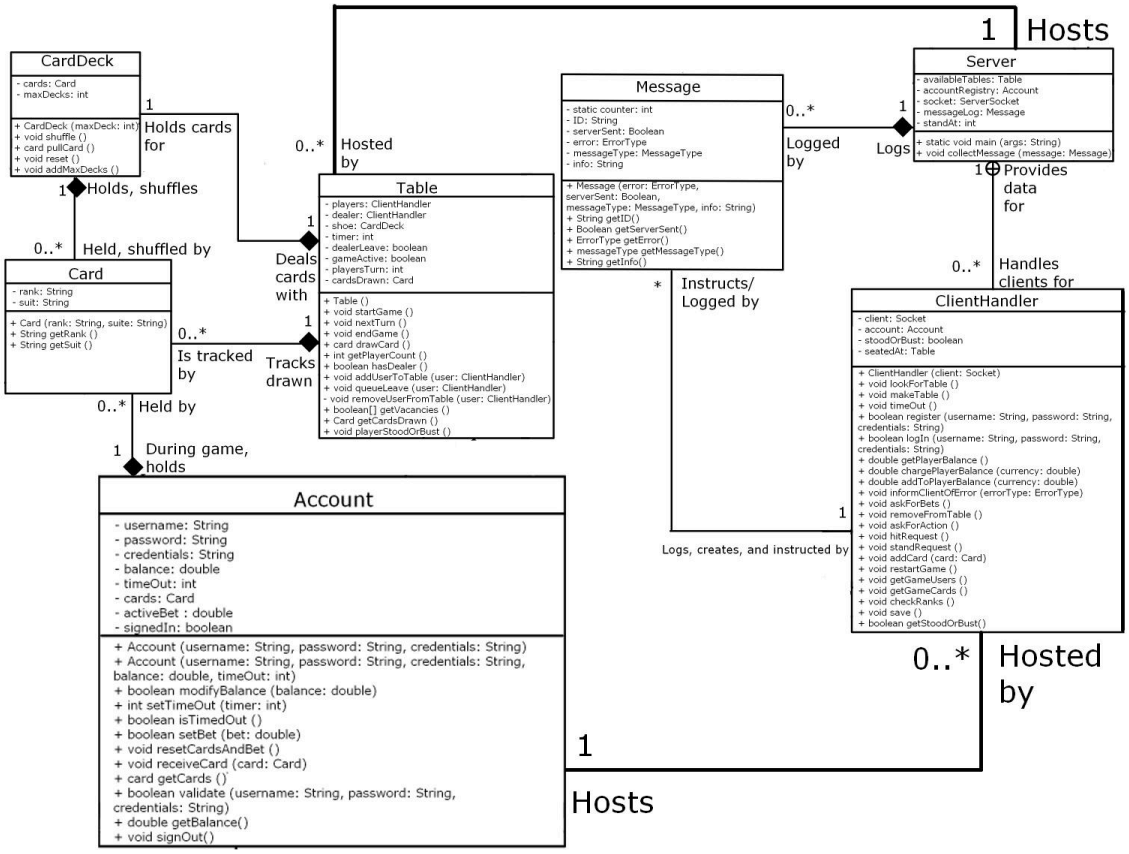
General UML Class Diagram:



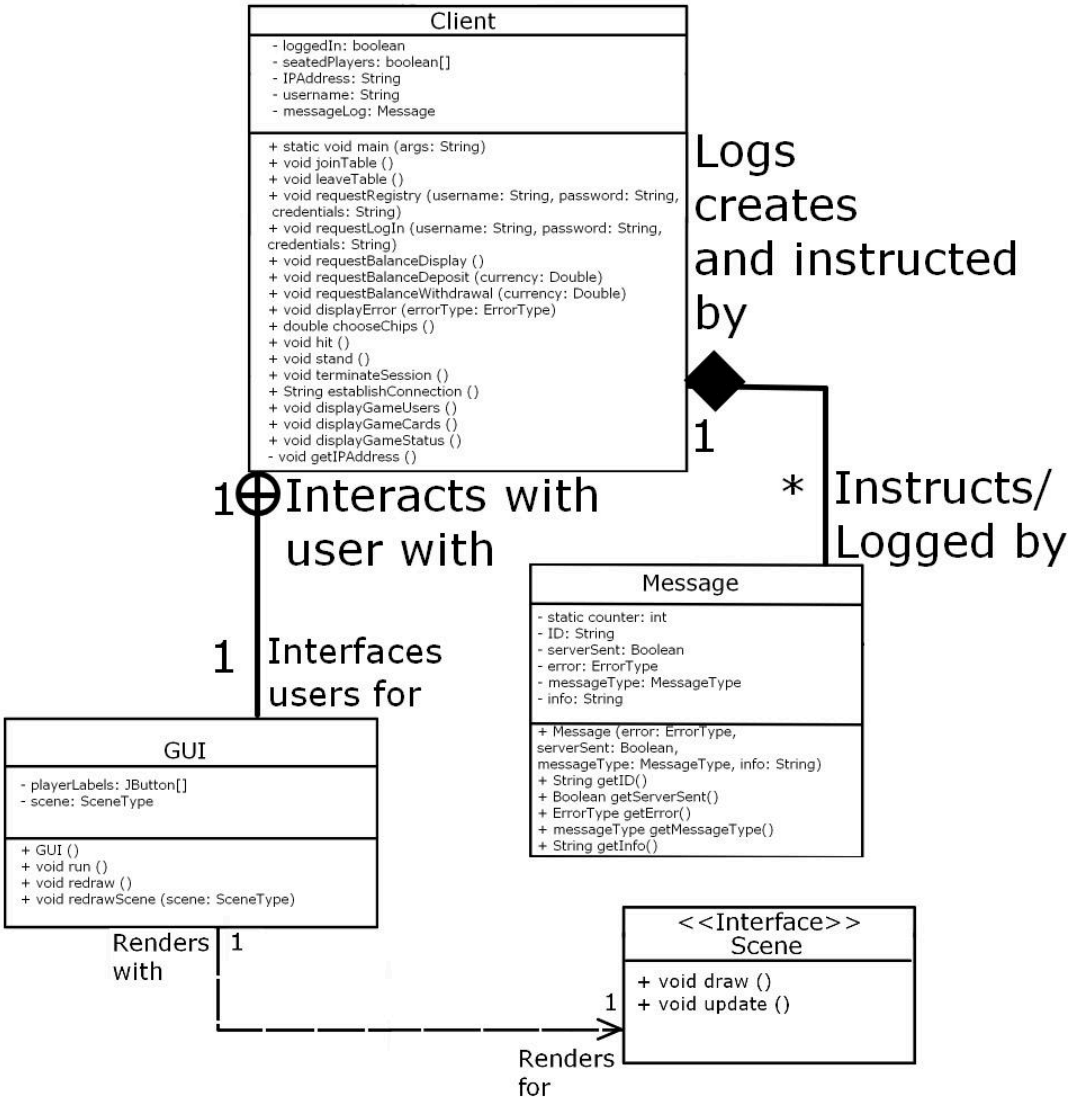
Simplified UML Class Diagram Displaying Packages:



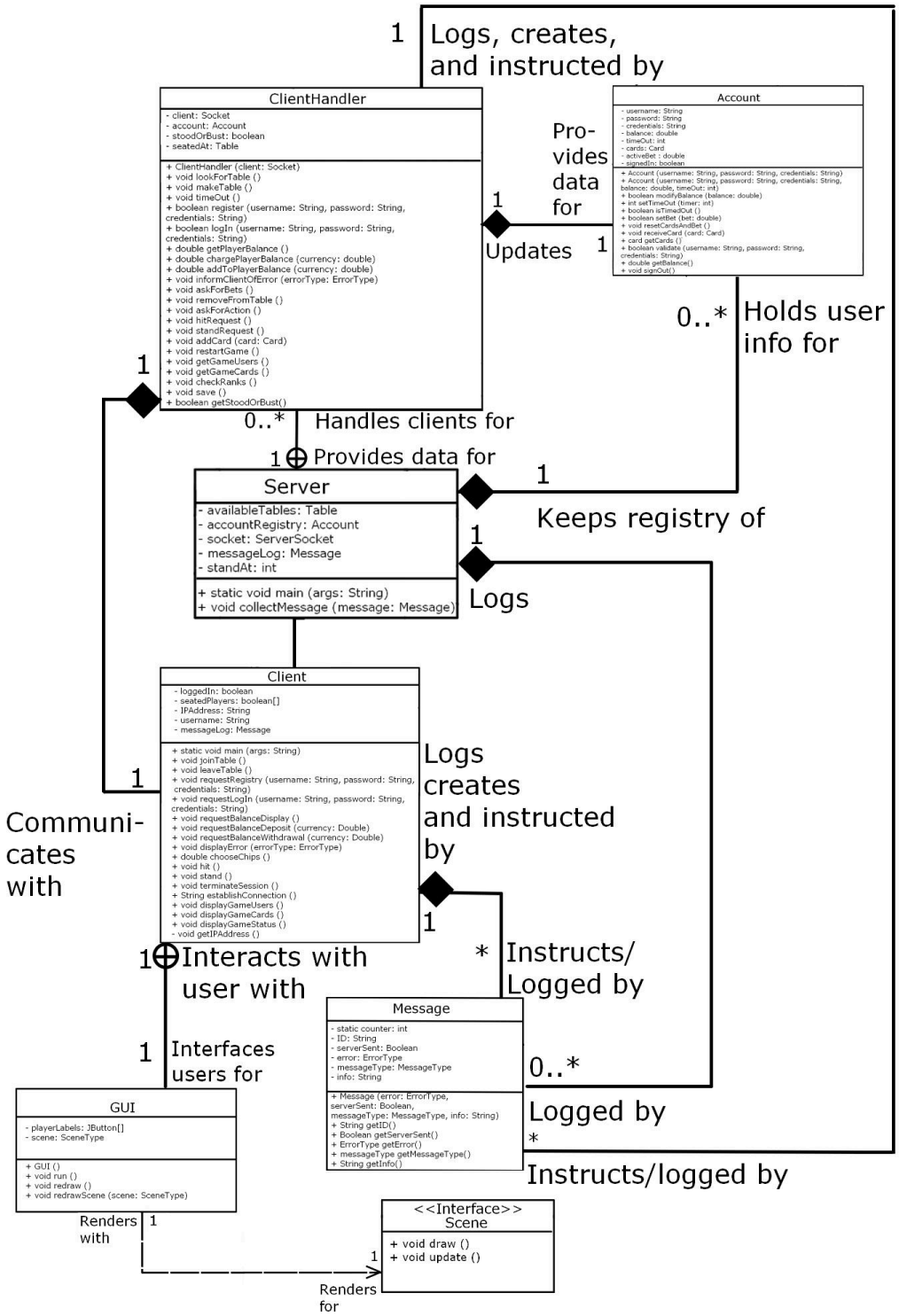
Server Module UML Class Diagram:



Client Module UML Class Diagram:



Dealer/Player Module UML Class Diagram:



Game Module UML Class Diagram:

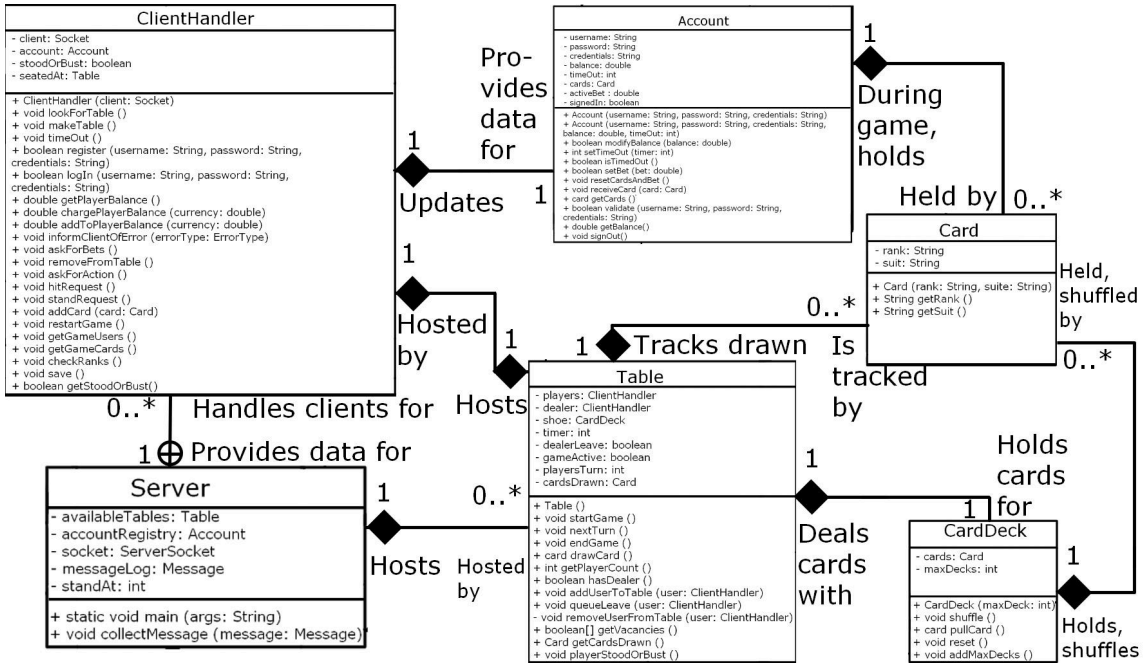
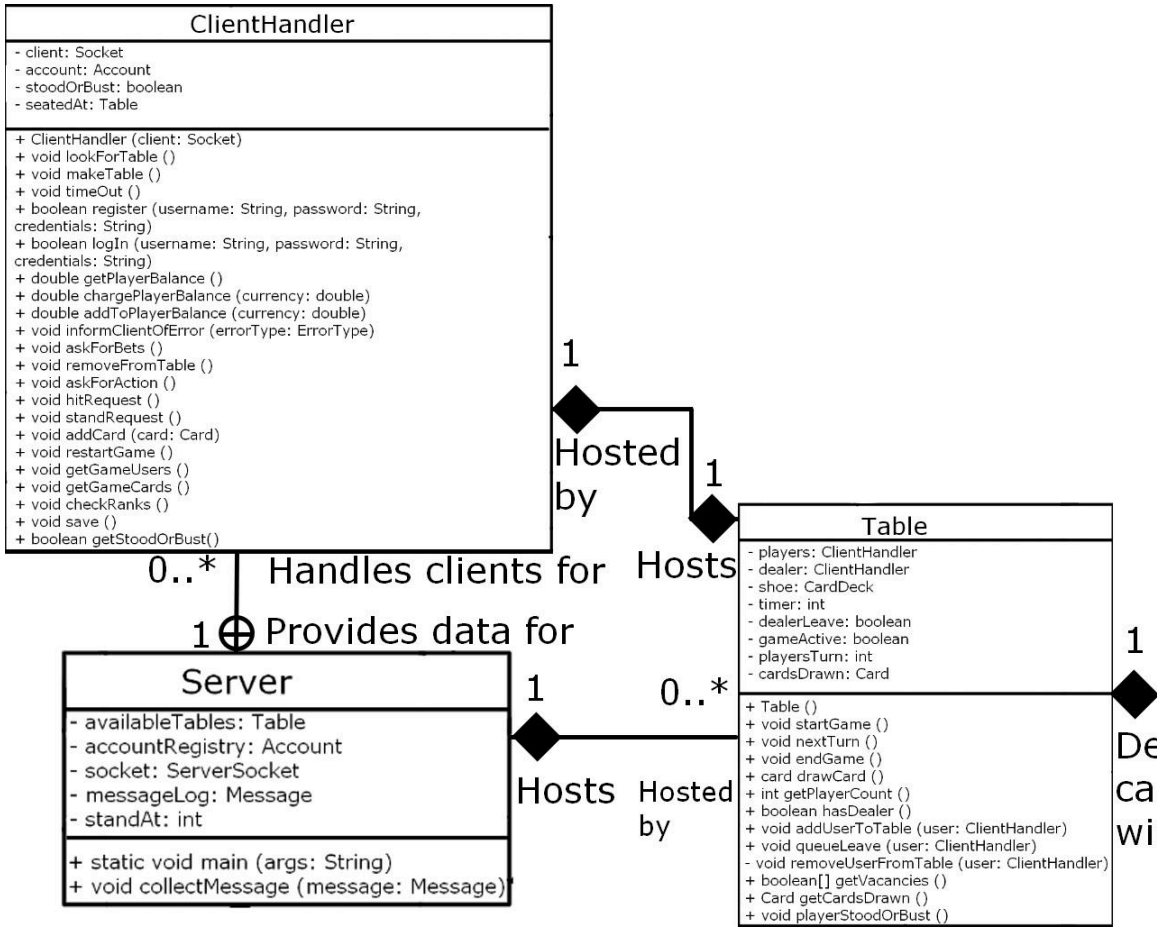
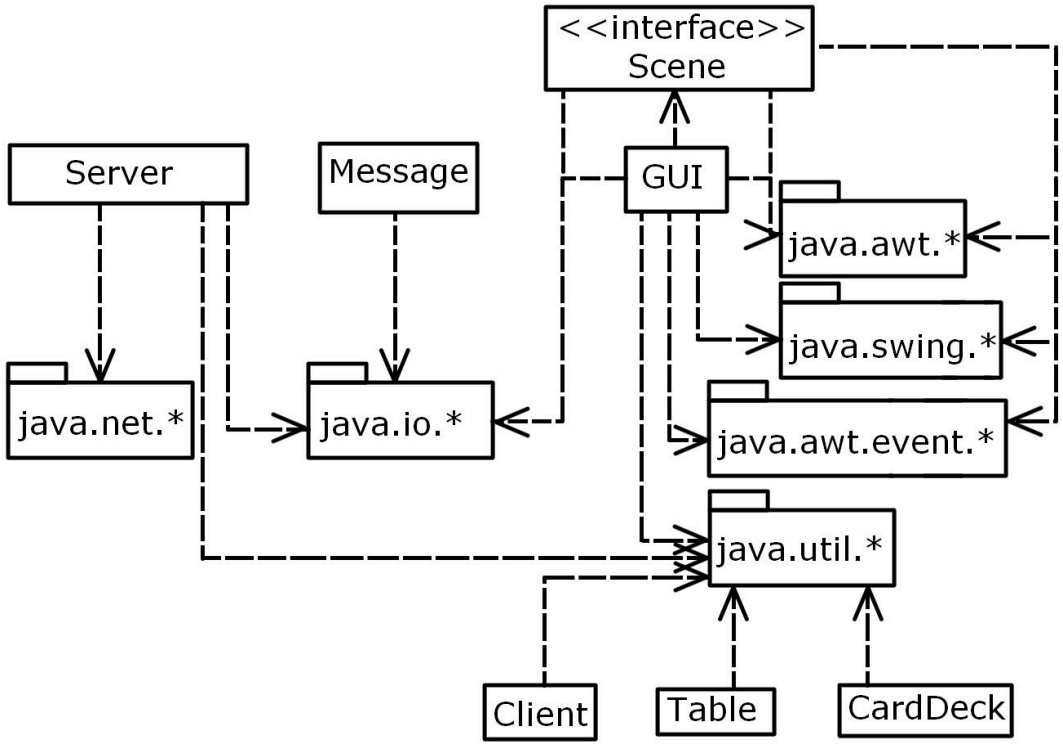


Table Module UML Class Diagram:

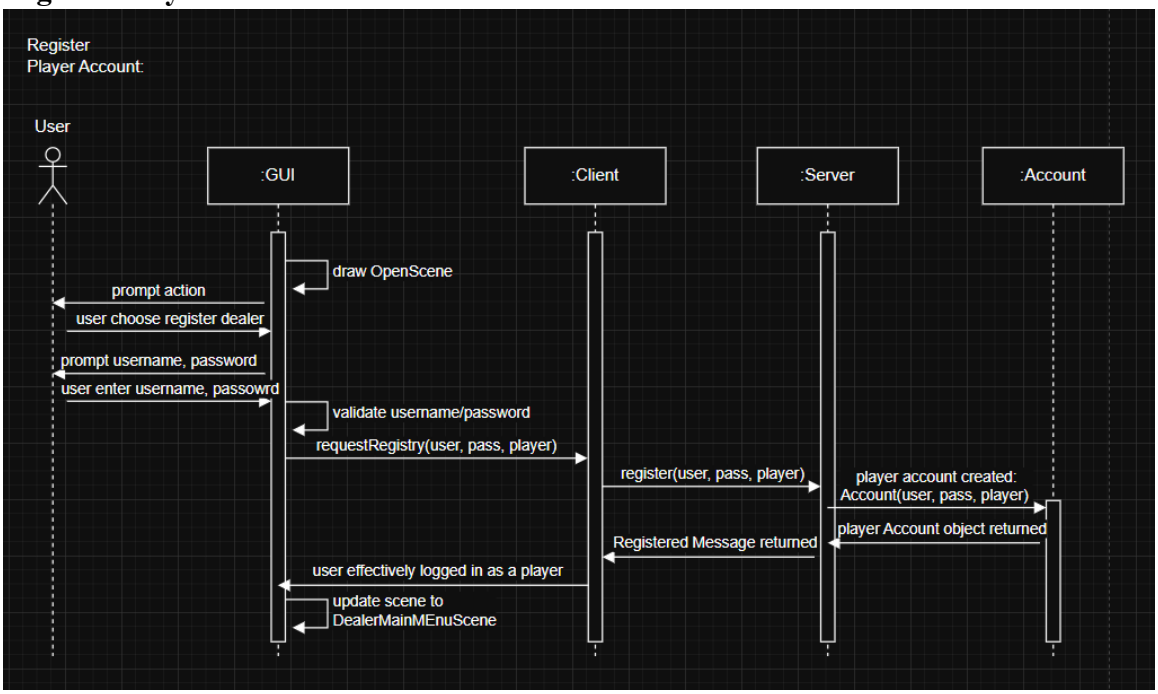


UML Class Diagram Library Dependencies:

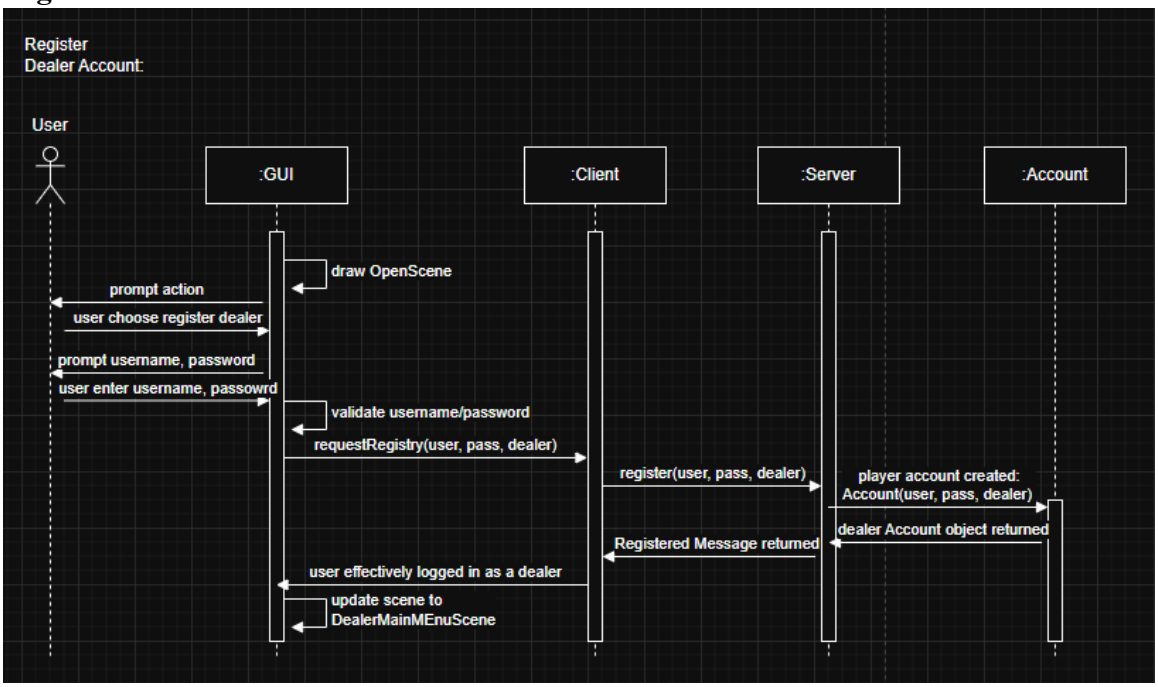


UML Sequence Diagrams

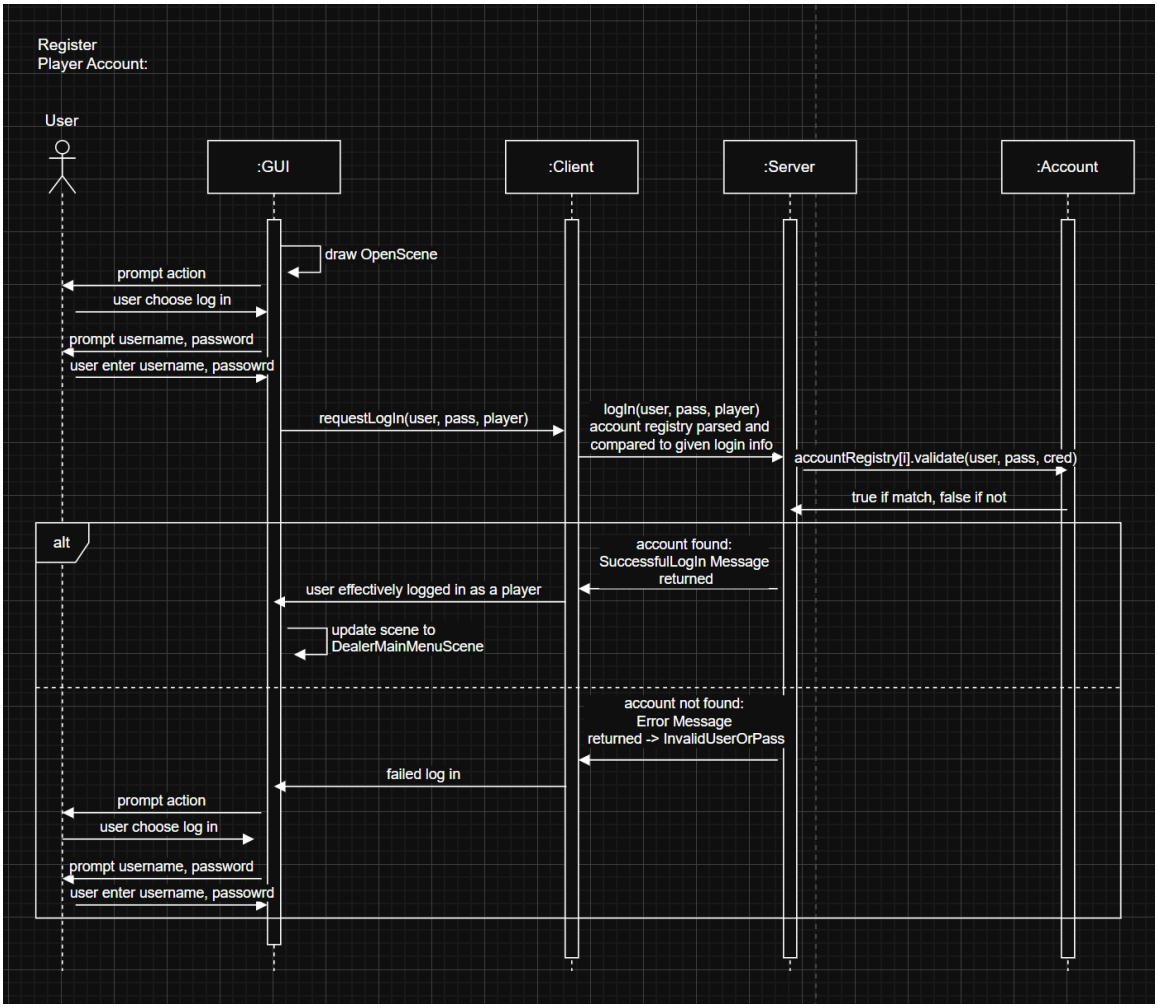
Register Player Account:



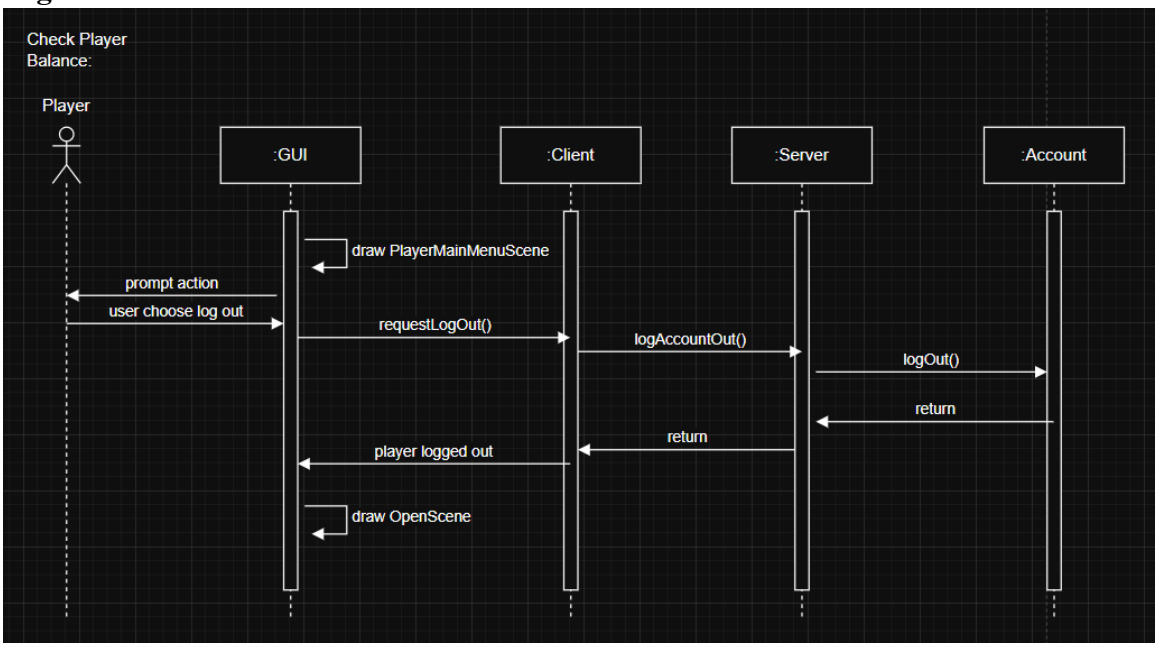
Register Dealer Account:



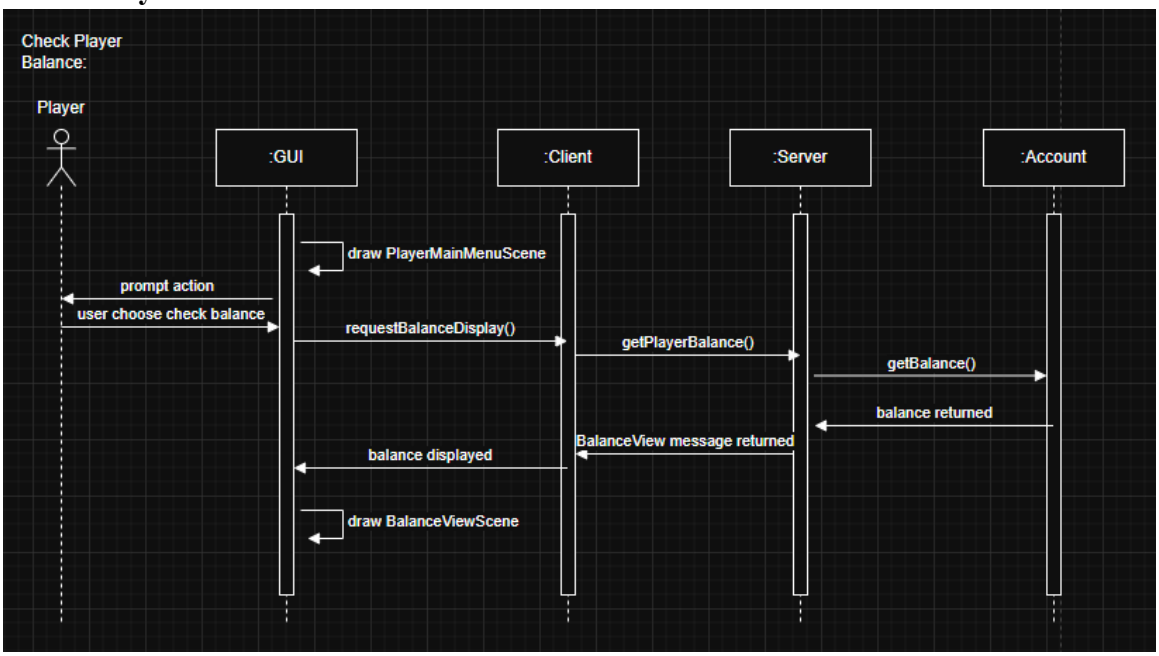
Log In


Register
Player Account:
User
:GUI
:Client
:Server
:Account
draw OpenScene
prompt action
user choose log in
prompt username, password
user enter username, passowrd
requestLogIn(user, pass, player)
logIn(user, pass, player)
account registry parsed and
compared to given login info
accountRegistry[i].validate(user, pass, cred)
true if match, false if not
alt
account found:
SuccessfulLogIn Message
returned
user effectively logged in as a player
update scene to
DealerMainMenuScene
account not found:
Error Message
returned -> InvalidUserOrPass
failed log in
prompt action
user choose log in
prompt username, password
user enter username, passowrd

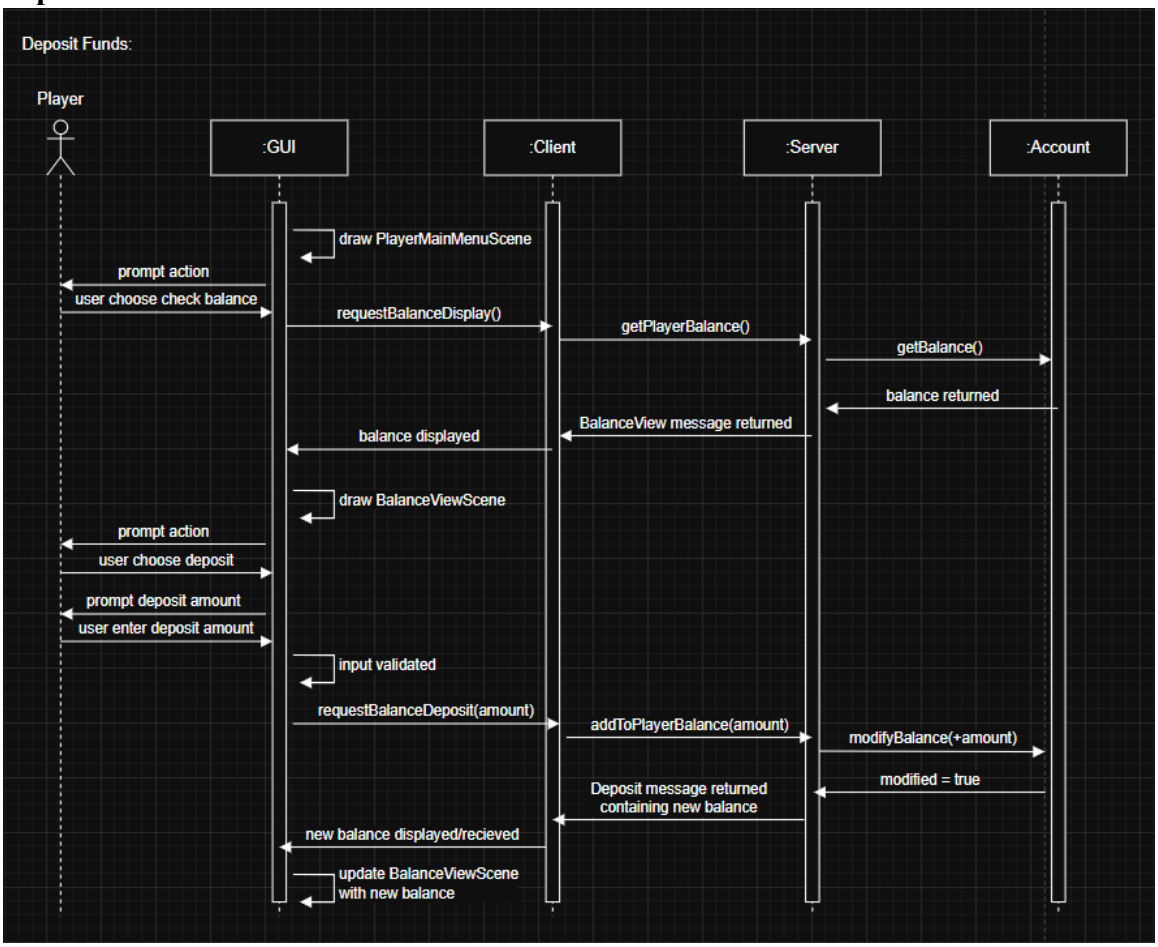
Log Out:



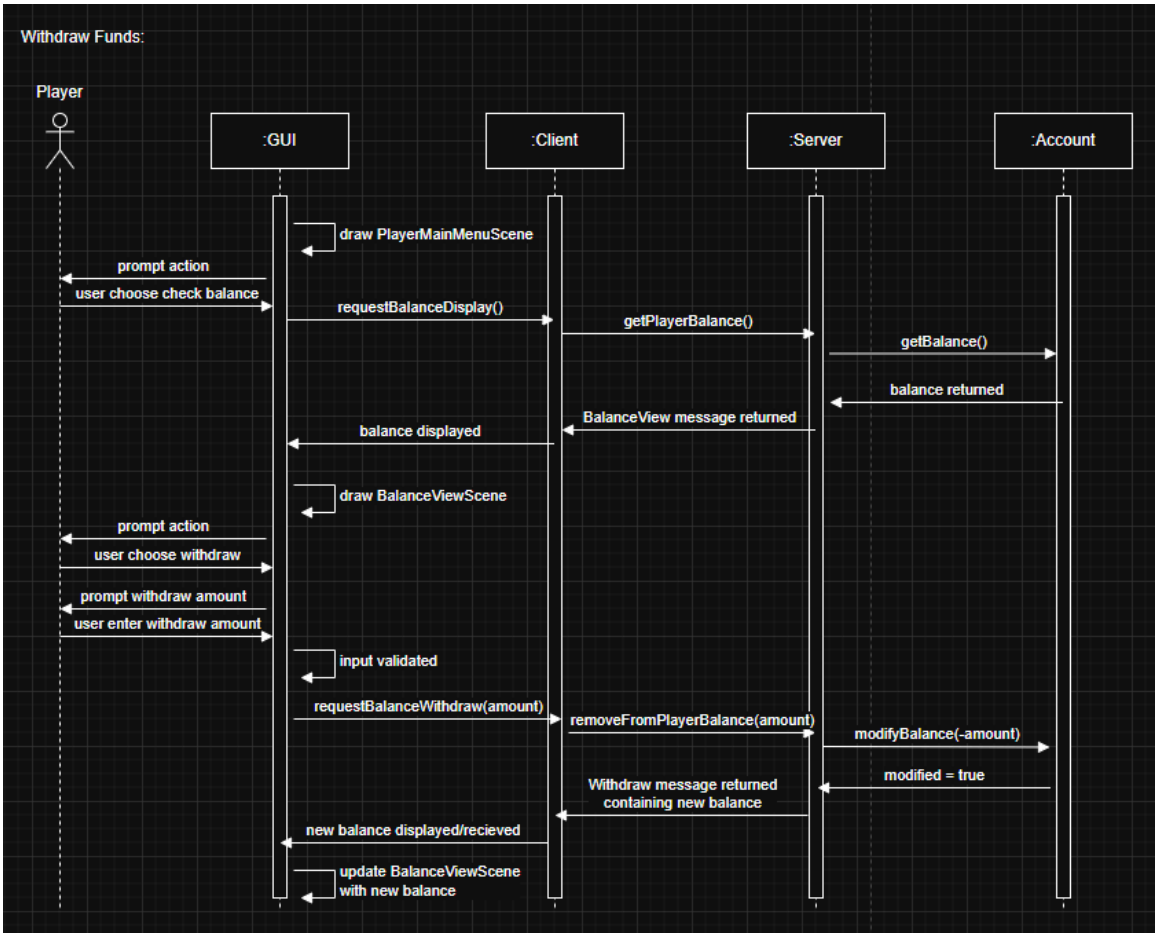
Check Player Balance:



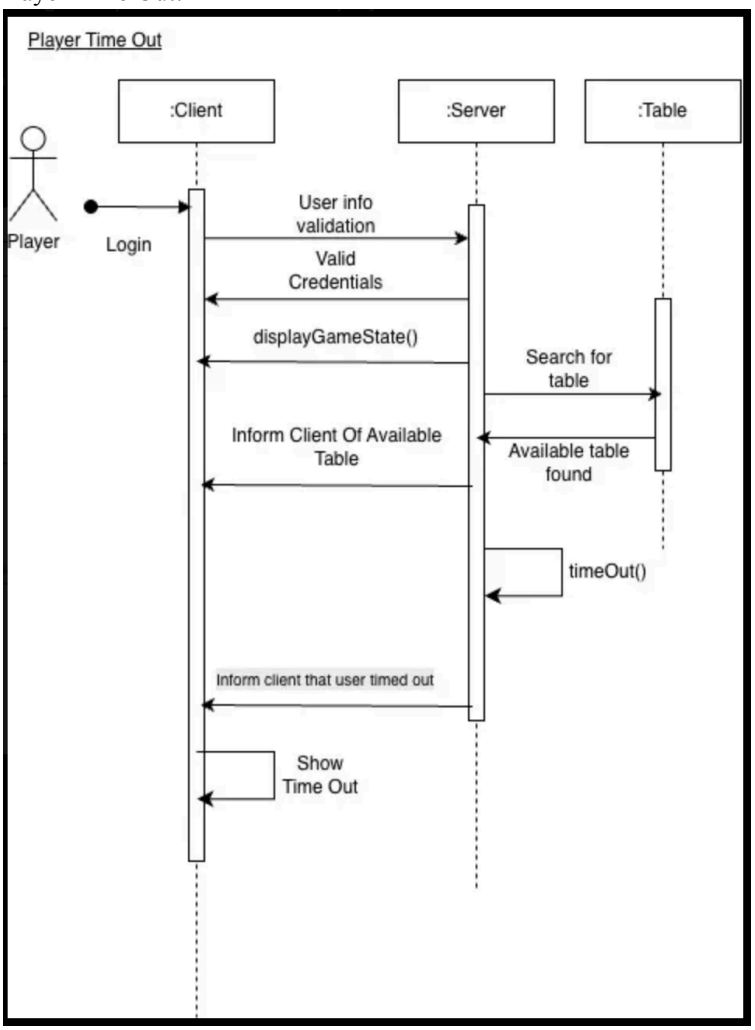
Deposit Funds:



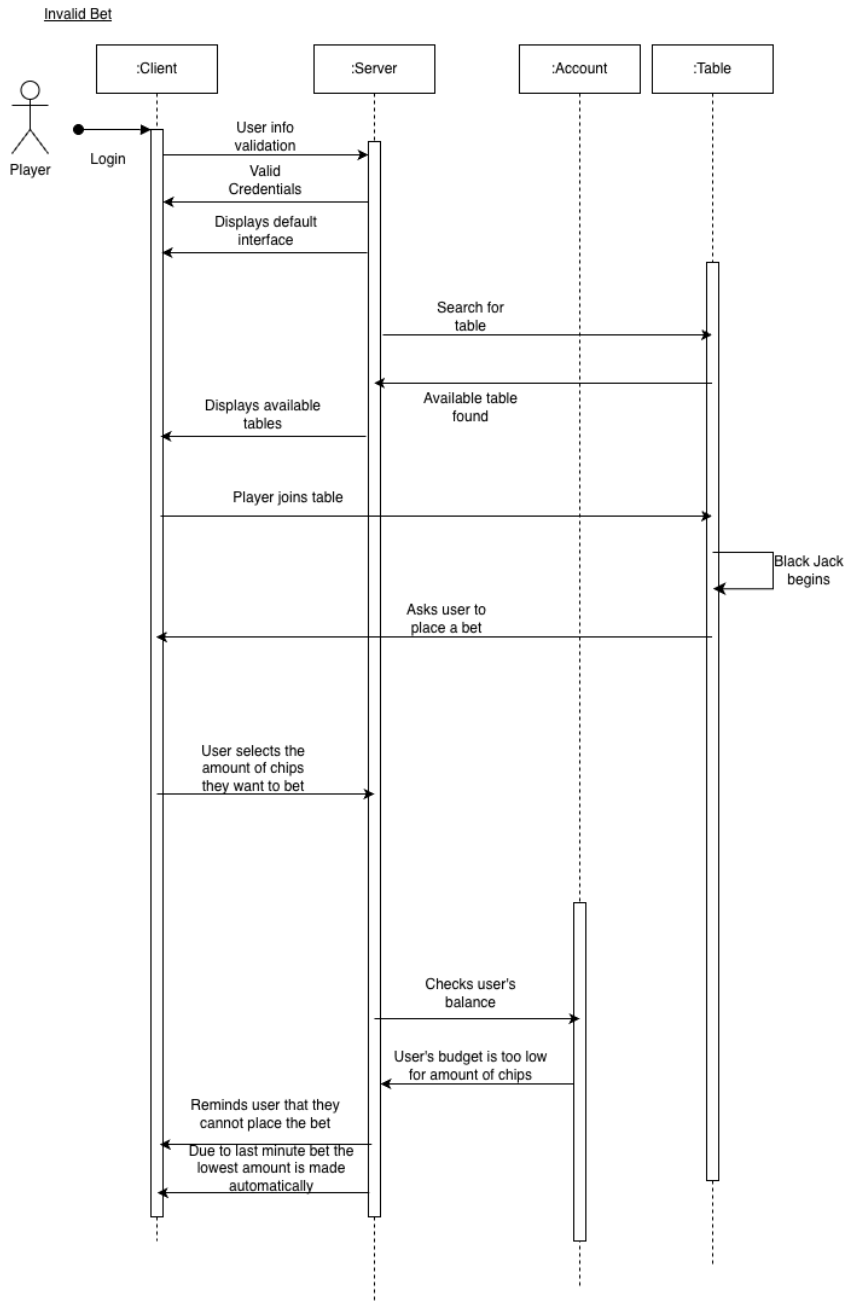
Withdraw Funds:



Player Time Out:

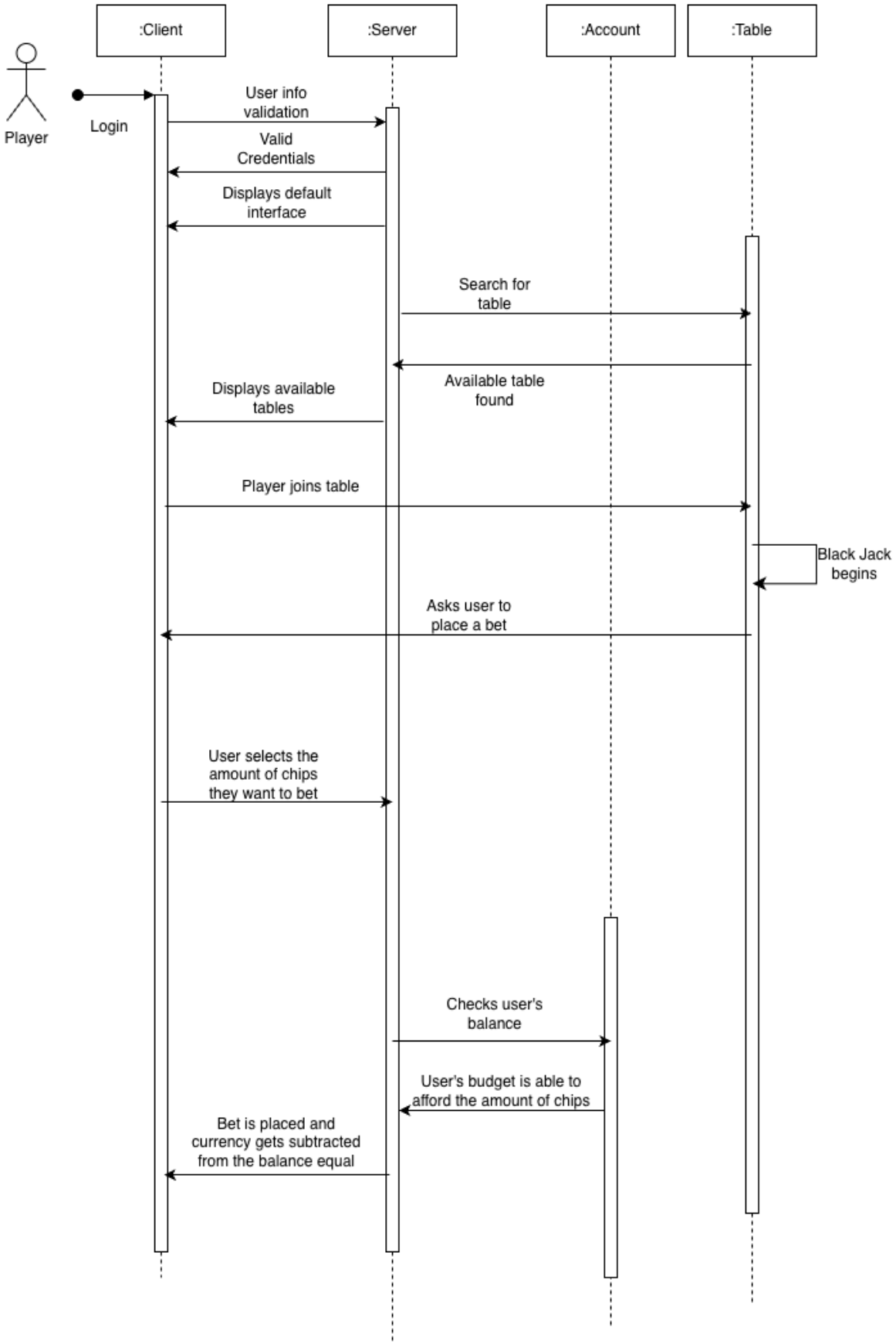


Player makes an invalid Bet:


Invalid Bet
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is too low for amount of chips
Reminds user that they cannot place the bet
Due to last minute bet the lowest amount is made automatically

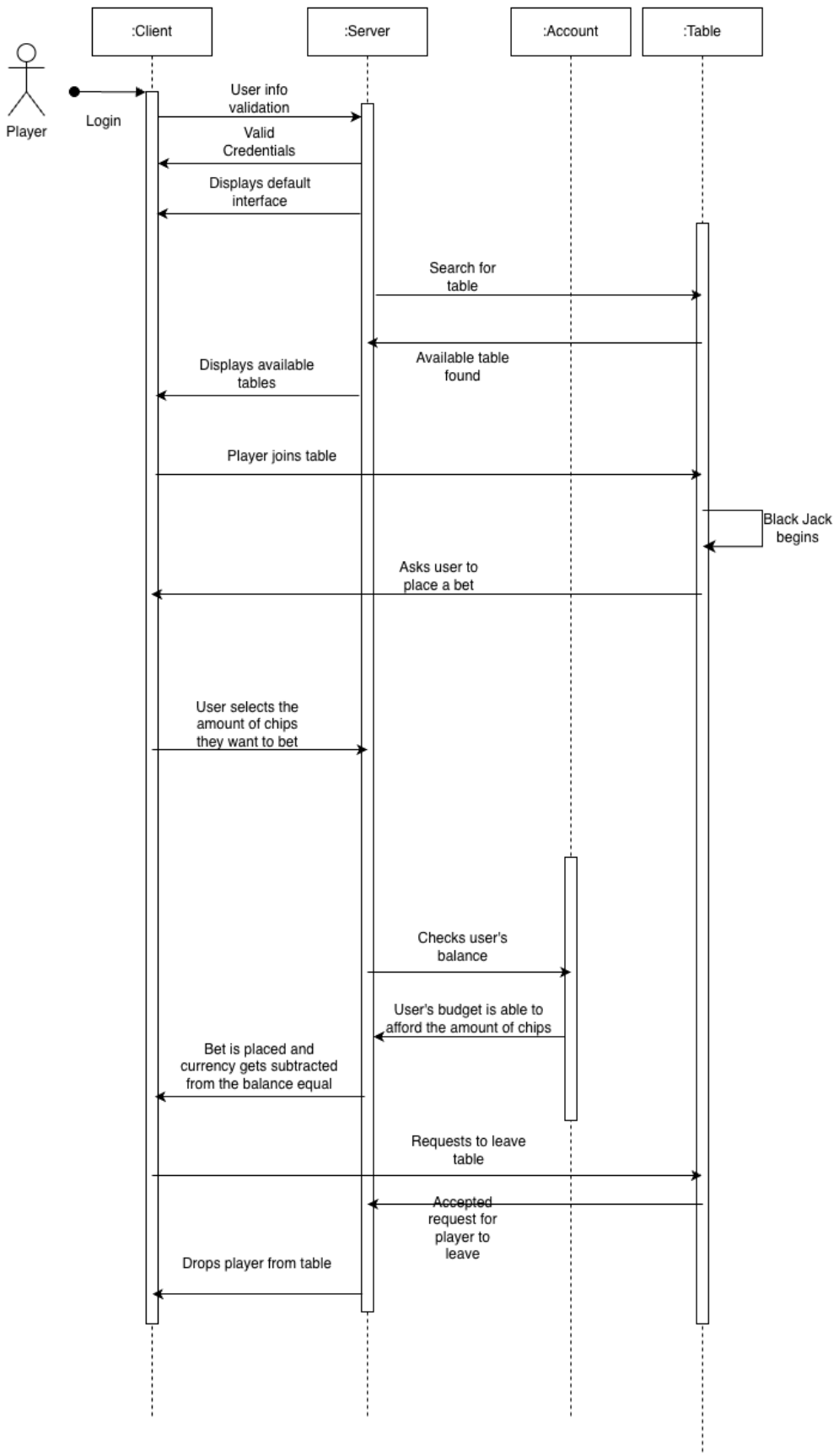
Bet:

Bet


:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal

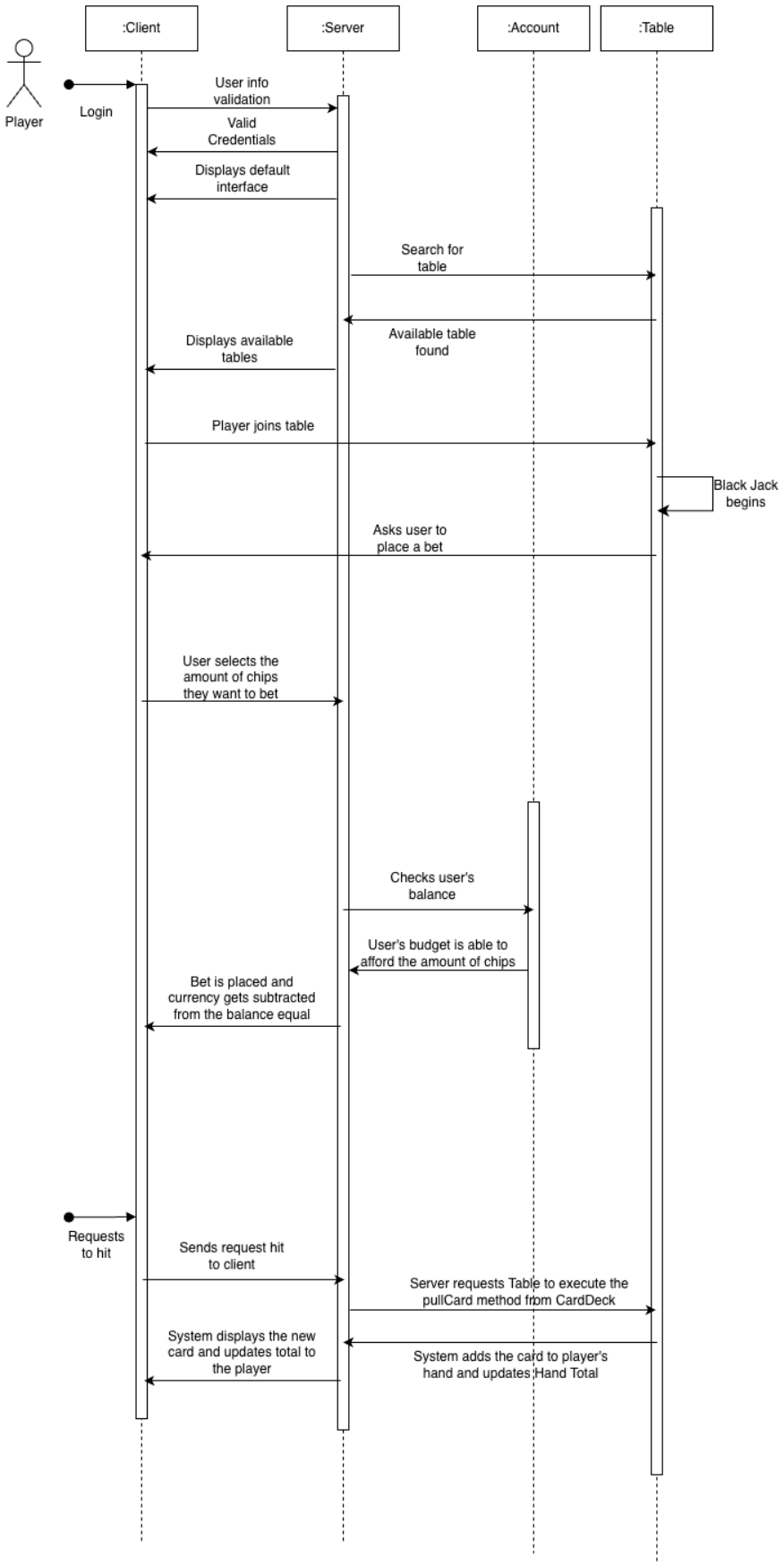
Player Leaves Table:

Player Leaves Table:

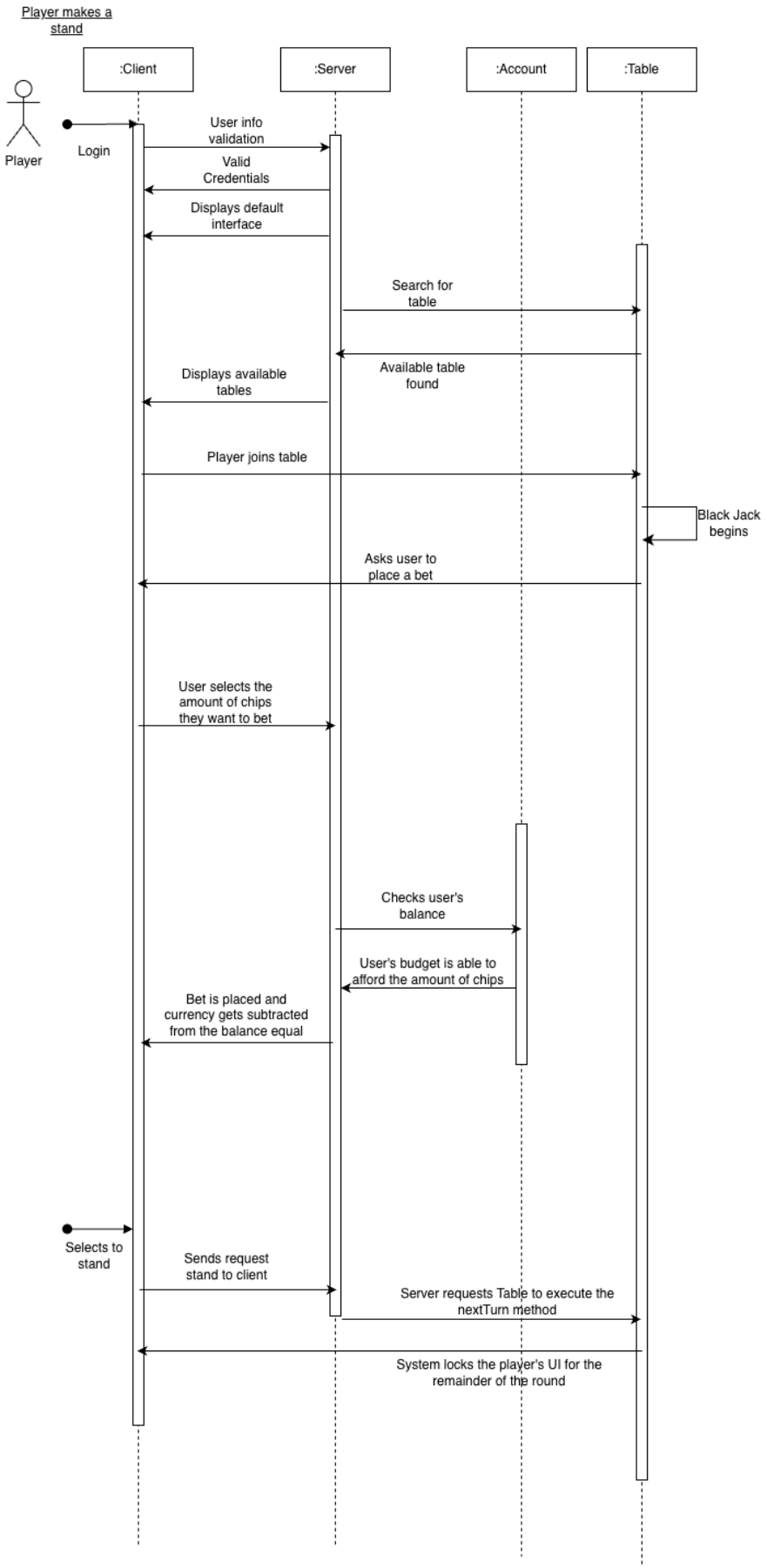


Players makes a hit:

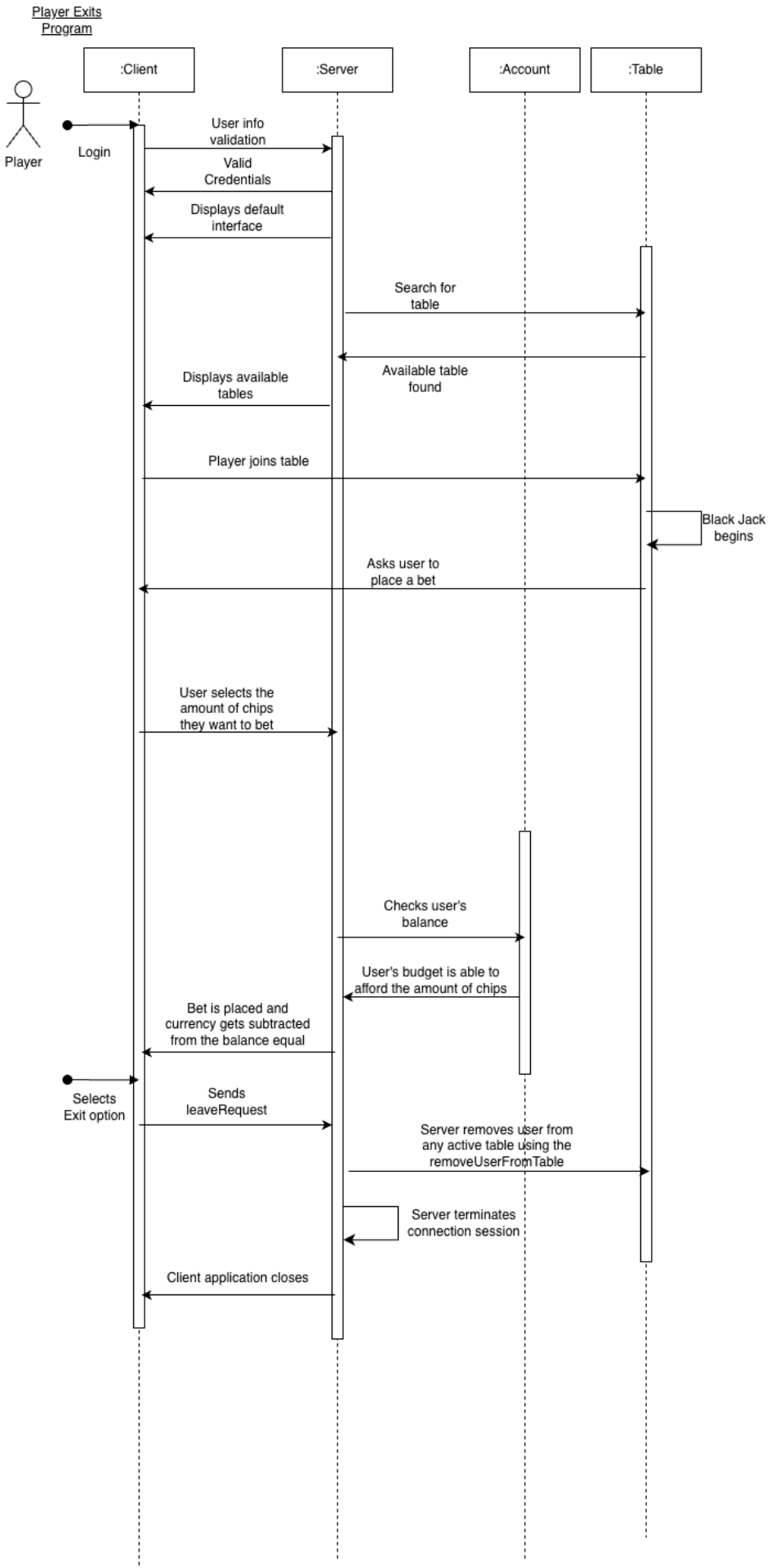
Player makes a hit



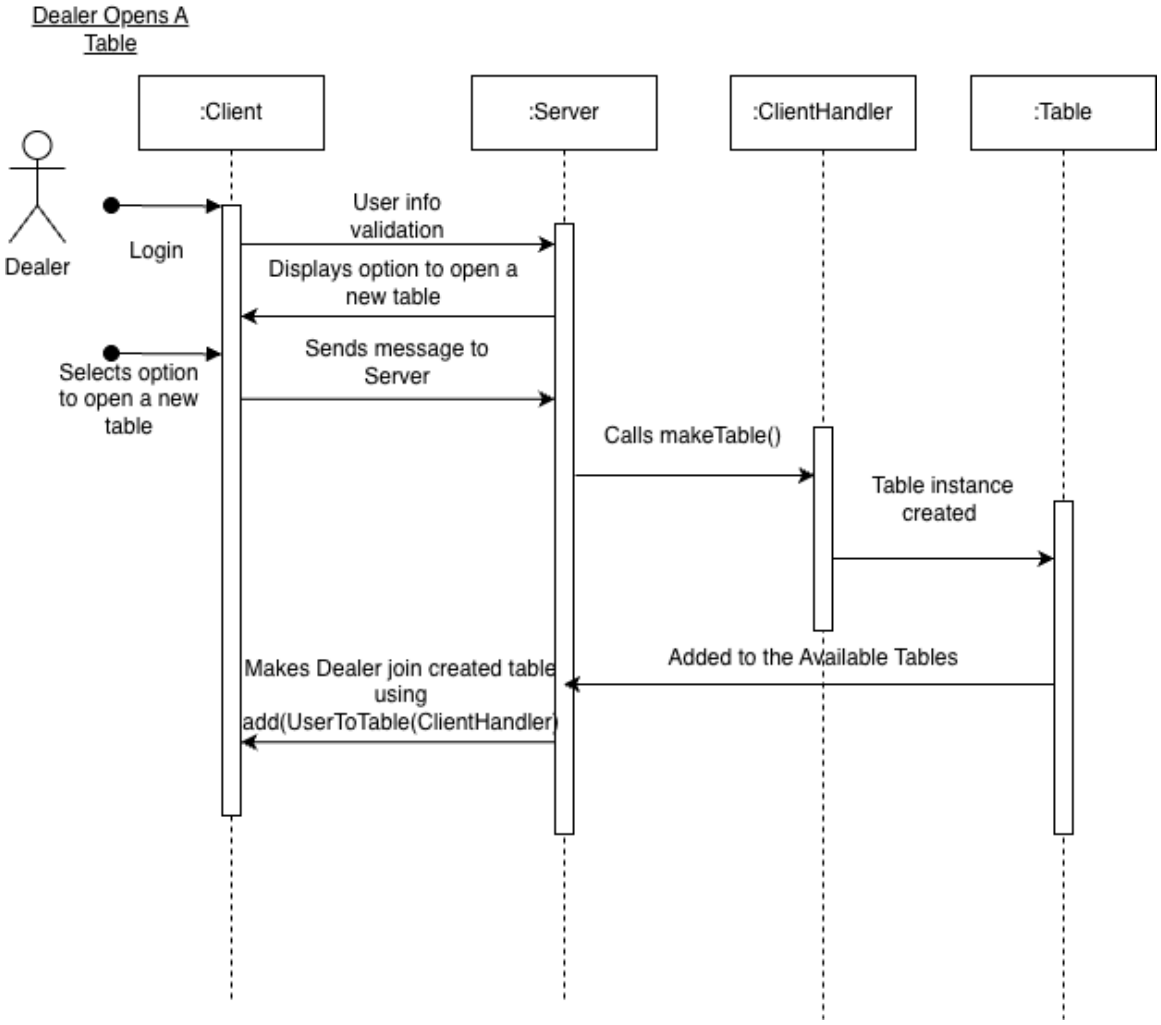
Player stands:


Player makes a stand
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal
Selects to stand
Sends request stand to client
Server requests Table to execute the nextTurn method
System locks the player's UI for the remainder of the round

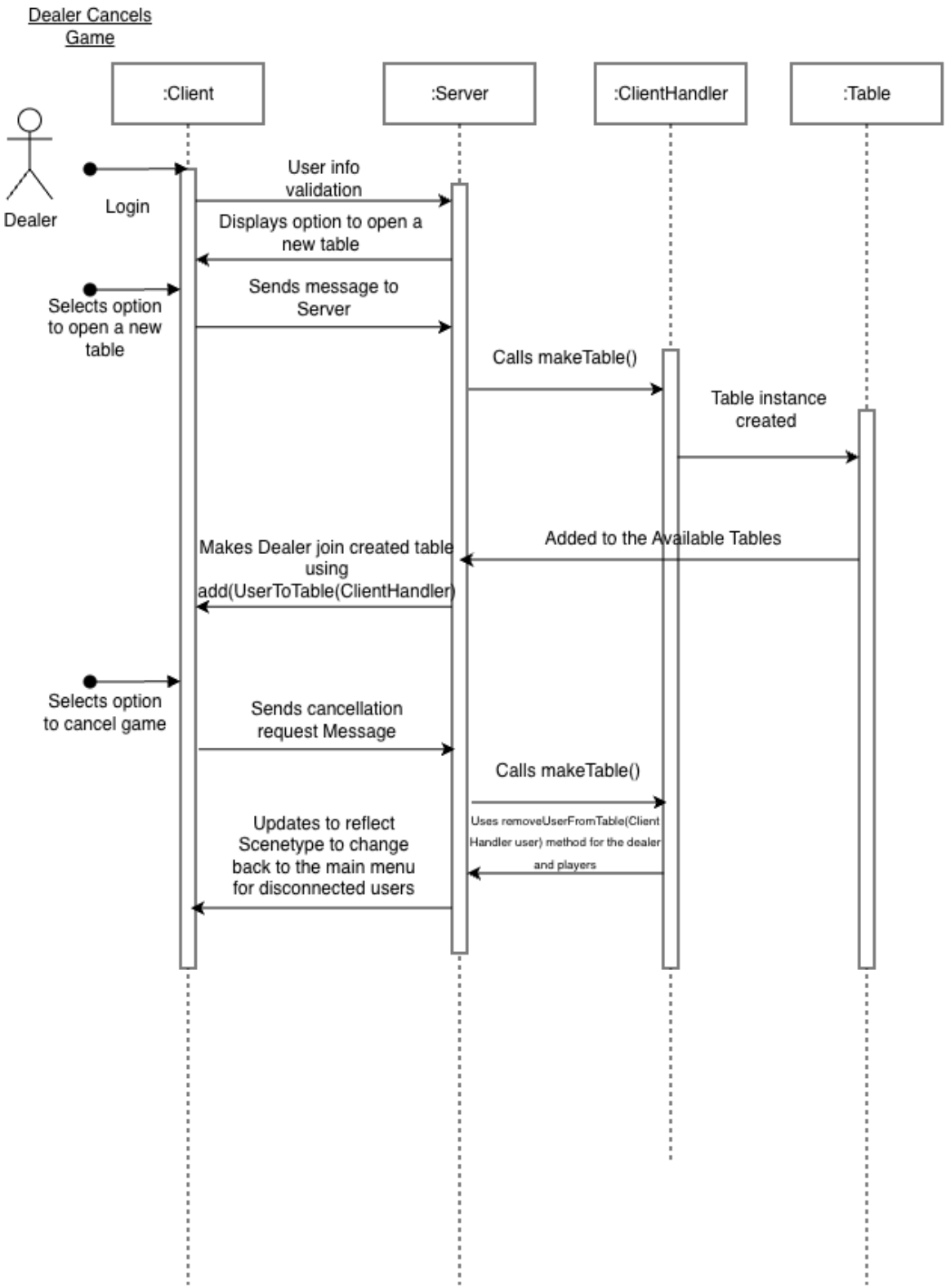
Player Exits Program:

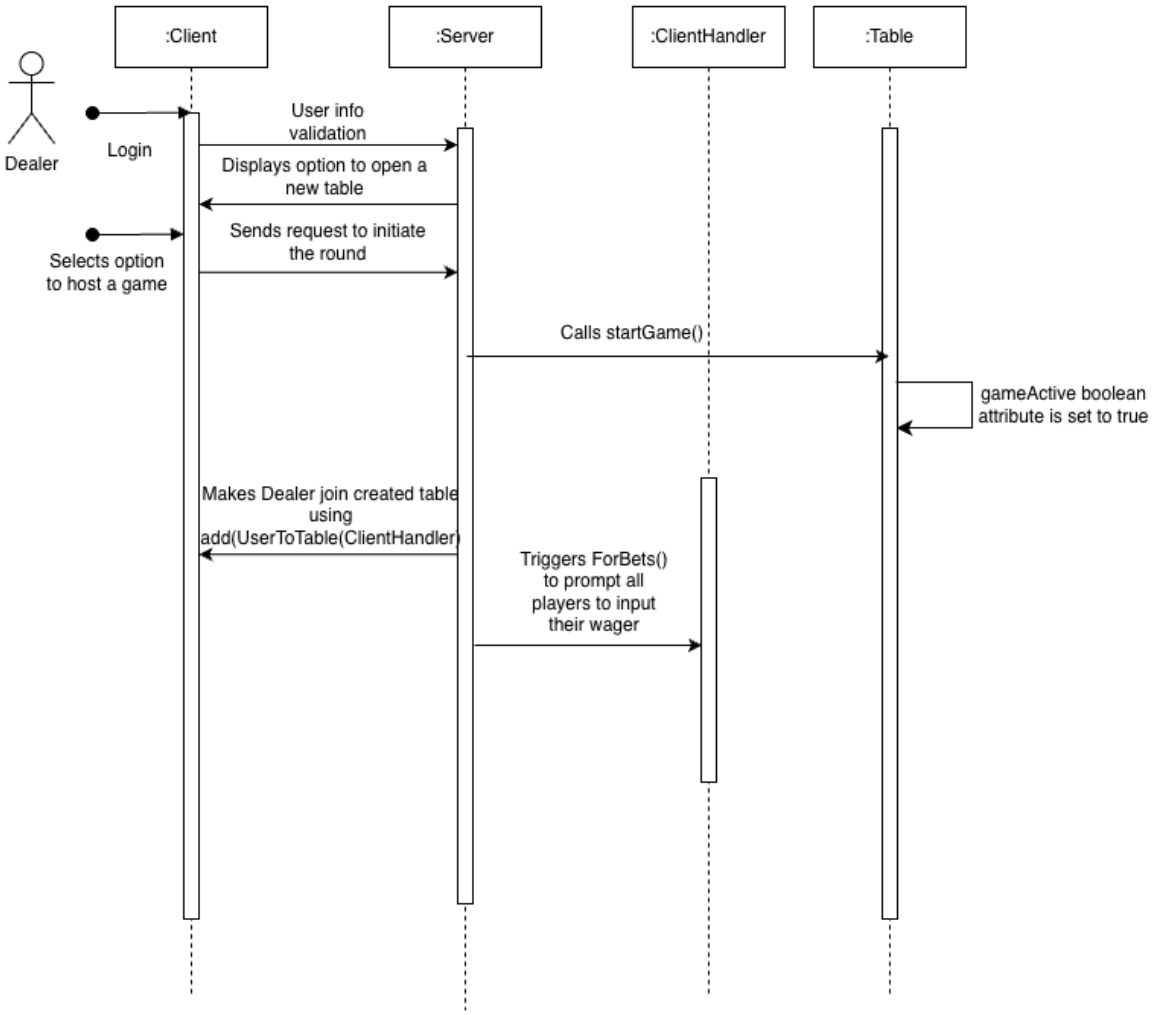

Player Exits Program
:Client
:Server
:Account
:Table
Player
Login
User info validation
Valid Credentials
Displays default interface
Search for table
Available table found
Displays available tables
Player joins table
Black Jack begins
Asks user to place a bet
User selects the amount of chips they want to bet
Checks user's balance
User's budget is able to afford the amount of chips
Bet is placed and currency gets subtracted from the balance equal
Selects Exit option
Sends leaveRequest
Server removes user from any active table using the removeUserFromTable
Server terminates connection session
Client application closes

Dealer Opens Table:



Dealer Cancels Game:



Dealer Hosts Game:

Dealer Hosts Game